OFFICE OF THE CHIEF SYSTEMS ARCHITECT

PLATFORM OPERATOR MANUAL

Opus Publica

Platform Operator Manual

The On-Call Engineer's Survival Guide for the RC2 Platform

Version 1.0

Issued 09 August 2026

Document Type	Operator Manual / Runbook Collection
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	On-call engineers, SREs, platform operators, duty engineers
Classification	Operational Reference — Required Reading for On-Call
Effective Date	09 August 2026
Companion Documents	Engineering Constitution (Vol I & II); Engineering Playbook Ch 13–16; Certification Checklist
Review Cadence	Quarterly, or after any SEV-1 incident
Document ID	OP-OPS-MAN-RC2-001

This manual is the operator's survival guide. When the pager goes off at 03:00, the operator reaches for this manual, not the 1,300-page Constitution. The Constitution defines the contracts; this manual executes them under pressure.

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Chapter 1 — Introduction

1.1 Opening Hook — The Operator's First 5 Minutes

> **03:00. The Pager.**
>
> The phone buzzes on the nightstand. The screen glows: PagerDuty — SEV-1 — INV-A-03 — Audit chain cryptographic verification FAILED. The operator's heart rate spikes. The Constitution is 1,300 pages long. The Playbook is 800 pages long. Neither is what the operator needs in the next five minutes. The operator needs the runbook for INV-A-03, the break-glass procedure, the communication template, and a short, blunt checklist of what to do, in what order, before the user-visible damage spreads. That is what this manual is.

The Platform Operator Manual exists because the Constitution and the Playbook, for all their authority and completeness, are not written for the operator at 03:00. The Constitution is written for the architect at 10:00 on a Tuesday. The Playbook is written for the engineer planning a release. The operator at 03:00 has three minutes to acknowledge the page, five minutes to triage, fifteen minutes to mitigate, and one hour to communicate to the Chief Systems Architect. None of those timelines are served by a 1,300-page document.

This manual compresses the operational kernel of the Constitution and the Playbook into a single, runbook-first reference. It assumes the operator has completed on-call eligibility training (Playbook Ch 13 §13.3), has access to the dashboards, the runbook repository, the break-glass credential store, and the incident bridge, and is operating within the authority matrix (Playbook Ch 14 §14.2). It does not re-teach the Constitution; it operationalizes it.

The operator's first five minutes are scripted. The operator acknowledges the page, reads the alert payload, opens the linked runbook, glances at the SLO dashboard to confirm the alert is real, glances at the deployment console to see whether a recent deploy is the proximate cause, and posts the first update to the incident channel — even if the update is only “investigating.” The Five-A pattern — Acknowledge, Assess, Act, Alert, Ascertain — is introduced in Playbook Ch 13 §13.8 and is the spine of every runbook in Chapter 6 of this manual.

1.2 Purpose, Audience, and Prerequisites

Purpose. This manual provides the on-call engineer with a single, authoritative, runbook-first reference for operating the Opus Publica RC2 platform in production. It defines the daily, weekly, monthly, and quarterly routines; the deployment, rollback, and hotfix procedures; the monitoring and alerting taxonomy; the incident-response runbooks for the eight most likely SEV-1 and SEV-2 scenarios; the break-glass procedures for when no other path exists; the disaster-recovery runbook; and the operator-

health policies that keep the rotation sustainable. It is normative where it prescribes procedures (RFC 2119 vocabulary) and informational where it describes context.

Audience. The primary audience is the on-call engineer (primary or secondary) carrying the pager for a production service. The secondary audience is the SRE or platform operator performing routine operations outside an incident, the duty engineer covering the platform during business hours, the Release Manager executing a deployment, and the Incident Commander coordinating a war room. Engineering leadership and new hires may read this manual as an orientation to how the platform is actually run.

Prerequisites. To use this manual operationally, the operator SHALL satisfy the on-call eligibility criteria of Playbook Ch 13 §13.3:

- **Rotation membership —** the operator SHALL be a named primary or secondary on the published rotation roster for the service in question.
- **Training —** the operator SHALL have completed on-call training, including Constitutional invariants, SLO management, the authority matrix, break-glass procedure, and the war-room convention.
- **Shadow shift —** the operator SHALL have completed at least one shadow shift with the previous primary, including a live or simulated incident walk-through.
- **Dashboard access —** the operator SHALL have read-only access to the Grafana dashboards, the Backstage scorecards, the SLO dashboard, the audit-log explorer, and the runbook repository.
- **Tool access —** the operator SHALL have SSO-authenticated, role-assumed access to kubectl, gh, vault (break-glass path), and the Crossref/ORCID/preservation control planes via the operator role.
- **Break-glass eligibility —** the operator SHALL have completed the break-glass training module and been granted the time-boxed credential checkout permission.
- **Communications —** the operator SHALL have a tested, working PagerDuty, Slack #ops, and incident-bridge client on the device that carries the pager.

1.3 Scope

This manual covers the operation of the Opus Publica RC2 platform in production. It covers deployment, monitoring, alerting, incident response, routine operations, break-glass, disaster recovery, and operator wellbeing. It does not cover editorial workflows, peer review, content production, or the business of running a publishing house; those are governed by the Editorial Constitution and are out of scope.

The manual covers the platform's four production subsystems: the publication pipeline (Crossref, ORCID, DOI minting, JATS rendering), the audit subsystem (cryptographic audit chain, INV-A-03), the traceability graph (Neo4j knowledge graph, eight integrity rules), and the CI/CD pipeline (GitHub Actions, Flux GitOps, Cosign signing, Rekor transparency log). The manual treats each subsystem as a separate operational unit with its own runbooks but assumes the operator understands their interdependence through the Constitutional invariants (Vol II Ch 26).

The manual is bound by the Constitution and the Playbook. Where this manual and the Playbook appear to conflict, the Playbook prevails; where the Playbook and the Constitution appear to conflict, the

Constitution prevails. This manual is the operational compression of those two documents, not a replacement for either.

1.4 Document Conventions

Severity levels. This manual uses the five-level severity taxonomy defined in Playbook Ch 16 §16.3 and reproduced in Chapter 5 of this manual. SEV-1 is constitutional-invariant violation or user-visible data loss; SEV-2 is critical SLO burn or major-subsystem outage; SEV-3 is degraded service or anomalous behaviour requiring investigation; SEV-4 is minor degradation or cosmetic; SEV-5 is informational.

Runbook structure. Every runbook in this manual follows the same eight-field structure: Trigger (what fires the alert or starts the procedure), Severity (the SEV level and the authority it invokes), Prerequisites (what the operator must have or must have done before starting), Steps (numbered, imperative, ordered), Verification (how the operator confirms the procedure succeeded), Escalation (who to page if the procedure fails or stalls), Post-Action (what must be filed within what window after the procedure), and Last-Reviewed (the date the runbook was last reviewed by the platform-operations team). Runbooks SHALL NOT be edited during an incident; an amendment is filed post-incident and reviewed in the next quarterly manual review.

Command formatting. Shell commands appear in Consolas code blocks. The prompt is omitted; commands are written as the operator types them. Placeholders appear in angle brackets, e.g. <namespace>. Long commands wrap. Example:

```
# Verify the constitutional health dashboard from the CLI
kubectl --context prod-eu -n opus-publica \
    get pod -l app=constitutional-health
opa eval -d policies/ -i input/<check>.json 'data.opus.constitutional.invariants'
```

Imperative voice. Procedures are written in the imperative: Run, Execute, Verify, Notify, Rollback, Declare. The operator is the implicit subject. Normative requirements use RFC 2119 vocabulary (MUST, SHALL, SHOULD).

Cross-references. The manual cross-references the Constitution (Vol I Ch N or Vol II Ch N), the Playbook (PB Ch N), the Certification Checklist (CC-INV-X-NN), and the SLO Book (SLO-NN). Where a runbook operationalizes a Constitutional rule, the rule ID is cited in the runbook header.

1.5 References

The operator SHALL treat the following references as the authoritative sources behind this manual. Where this manual is silent or ambiguous, the operator SHALL consult the reference, not improvise.

Reference	Source	Authority
Engineering Playbook Ch 13 — On-Call and Operational Ownership	Opus Publica internal; PB Ch 13	Normative for rotation, eligibility, handoff, no-on-call-no-release.

Engineering Playbook Ch 14 — Operational Governance Execution	Opus Publica internal; PB Ch 14	Normative for the authority matrix, break-glass, legal-removal.
Engineering Playbook Ch 15 — SLO Management	Opus Publica internal; PB Ch 15	Normative for SLO dashboard, burn-rate, error-budget, SLO amendment.
Engineering Playbook Ch 16 — Incident Command	Opus Publica internal; PB Ch 16	Normative for severity, IC role, war room, postmortem.
Site Reliability Engineering	Google; https://sre.google/sre-book/table-of-contents/	Authoritative on SLOs, error budgets, toil, incident response.
The Site Reliability Workbook	Google; https://sre.google/workbook/table-of-contents/	Authoritative on SLO implementation, alerting, on-call.
NIST SP 800-61 Rev. 2 — Computer Security Incident Handling Guide	NIST; https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final	Authoritative on incident lifecycle, communication, coordination.
RFC 2119 — Key words for use in RFCs to Indicate Requirement Levels	IETF; https://www.rfc-editor.org/rfc/rfc2119	Authoritative for MUST, SHALL, SHOULD vocabulary in this manual.
Volume II Ch 26 — Invariants Catalog	Opus Publica internal; Const. Vol II Ch 26	Normative source of INV-I, INV-A, INV-T, INV-D, INV-S, INV-L invariants.
Volume II Ch 32 — Production SLO Book	Opus Publica internal; Const. Vol II Ch 32	Normative source of the ten SLOs and three SLAs.
Volume II Ch 33 — Disaster Recovery	Opus Publica internal; Const. Vol II Ch 33	Normative source of RPO 15 min / RTO 1 hour and the DR plan.
Volume II Ch 34 — Operational Governance	Opus Publica internal; Const. Vol II Ch 34	Normative source of the authority matrix (OP-GOV-001 … OP-GOV-014).

Chapter 2 — The Operator's Toolkit

Before the operator can run a runbook, the operator must have the tools, the access, the channels, and the rotation context. This chapter inventories the toolkit, documents the access model, lists the communication channels, and specifies the on-call rotation. The operator SHALL verify this toolkit is reachable at the start of every shift (Appendix A, items H-01 through H-05).

2.1 The Dashboard Stack

The operator's primary situational awareness comes from three dashboards, each of which answers a different question. The operator SHALL have all three open during any incident and SHALL keep the Constitutional Health dashboard visible during routine shifts.

Dashboard	URL / Location	Question Answered	Owner
Grafana: Constitutional Health	grafana.ops.opuspublica.com/d/constitutional-health	Are all 47 invariants (INV-I/A/T/D/S/L) green right now?	Platform Operations
Grafana: SLO Burn	grafana.ops.opuspublica.com/d/slo-burn	Are any of the ten SLOs burning error budget faster than allowed?	SRE team
Grafana: Audit Chain	grafana.ops.opuspublica.com/d/audit-chain	Is INV-A-03 (cryptographic chain) verifiable end-to-end?	Security team
Grafana: Traceability Graph	grafana.ops.opuspublica.com/d/traceability	Are all eight traceability integrity rules passing? Are there orphans?	Knowledge-graph team
Grafana: CI/CD Health	grafana.ops.opuspublica.com/d/cicd	Is GitHub Actions healthy? Is Flux syncing? Is Rekor accepting entries?	Release Engineering
Backstage: Scorecards	backstage.opuspublica.com/scorecards	Does each service meet its Production Readiness scorecard?	Developer Experience
PagerDuty: Live Incidents	opuspublica.pagerduty.com	What is paging right now? Who is primary? Who is secondary?	Platform Operations
Status Page (internal)	status-internal.opuspublica.com	What is the user-visible status of the platform?	Platform Operations

Convention. The Constitutional Health dashboard is the operator's home. The operator SHALL bookmark it, SHALL keep it as the browser's first pinned tab, and SHALL glance at it at the start of every shift, before every deployment, and after every alert. If the dashboard itself is down, the operator SHALL treat that as a SEV-2 incident and follow the runbook in §6.7.

2.2 The Command-Line Tools

The operator SHALL have the following CLI tools installed, authenticated, and tested at the start of every shift. A missing or broken tool is a shift-readiness failure and SHALL be remediated before the operator takes the pager.

Tool	Purpose	Authentication	Test Command
kubectl	Kubernetes control plane operations; pod inspection; log retrieval.	SSO → role assumption via kubelogin.	kubectl --context prod-eu get ns
opa	Open Policy Agent; run invariant checks locally; pre-flight policies.	None (local binary).	opa version
conftest	Rego policy testing; verify manifests before deploy.	None (local binary).	conftest --version
neo4j-admin	Neo4j administration; integrity check; backup verification.	Vault-stored credentials, time-boxed.	neo4j-admin database check-store
cosign	Sigstore Cosign; verify container signatures in production.	OIDC + Sigstore (no long-lived key).	cosign verify --key prod.pub <image>
rekor-cli	Sigstore Rekor; verify transparency-log entries.	OIDC.	rekor-cli search --artifact <hash>
gh	GitHub CLI; merge PRs; trigger workflows; check Action runs.	SSO → gh auth login.	gh auth status
vault	HashiCorp Vault; check out break-glass credentials.	SSO + OIDC role.	vault token lookup
flux	Flux GitOps CLI; reconcile; suspend; resume; check sync.	In-cluster identity + kubectl proxy.	flux --context prod-eu get kustomization
jq	JSON parsing; essential for alert payload inspection.	None.	jq --version

aws-cli / gcloud	Cloud control plane (storage, DNS, IAM) for DR and break-glass.	SSO → role assumption.	aws sts get-caller-identity

All CLIs SHALL be the version pinned in the operator-kit repository. The operator SHALL run `make verify-kit` at the start of every shift; the Makefile verifies versions, auth, and connectivity. A failure SHALL be remediated before the pager is acknowledged.

2.3 The Access Model

The operator accesses production through three layers: SSO, role assumption, and break-glass. The model is least-privilege by default, time-boxed by exception, and fully audited at all times.

2.3.1 Single Sign-On (SSO)

The operator authenticates once via the corporate IdP (Okta). SSO is the identity root. All role assumptions, all Vault checkouts, all break-glass requests, and all dashboard sessions derive from the SSO identity. There is no shared account, no service-account password shared between humans, and no long-lived API token for production access. SSO MFA SHALL be a hardware token (FIDO2); TOTP is permitted only as a fallback when the hardware token is lost, and SHALL be replaced within 24 hours.

2.3.2 Role Assumption

From the SSO identity, the operator assumes one of three roles:

Role	Scope	Duration	Granted By
operator-read	Read-only: kubectl get, logs, describe; Grafana; Backstage.	8-hour session, renewable.	Automatic on shift start.
operator-deploy	Deploy via Flux (apply manifests), toggle feature flags.	4-hour session.	Rotation Lead; recorded in rotation log.
operator-break-glass	Direct production mutation; bypasses Gate.	30 minutes; non-renewable.	CSA approval per invocation.

Every role assumption is logged in the audit chain (INV-A-01, INV-A-03). The operator SHALL NOT share assumed-role sessions; the SSO identity SHALL be traceable to a single human for every action in the audit log.

2.3.3 Break-Glass

Break-glass is the only path to direct production mutation (DB writes, manual deploys, direct API calls that bypass the Gate). Break-glass is governed by Playbook Ch 14 §14.5 and Chapter 7 of this manual.

The credential is time-boxed (30 minutes), checked out from Vault, fully audit-logged, and reviewed by the Tech Lead within 24 hours of invocation. Break-glass SHALL NOT be used for routine operations; it SHALL be used only when SEV-1 impact has no other path or when the CI/CD pipeline is down during an incident.

2.4 The Communication Channels

Channel	Purpose	Audience	Cadence
PagerDuty	Page the on-call; track acknowledgement; escalate to secondary.	Primary + secondary on-call.	Real-time; 5-min ack SLA.
Slack #ops	Operational chatter; alert summaries; routine coordination.	All operators + SREs + Release Managers.	Continuous.
Slack #incident-<id>	War-room channel per incident; timeline + decisions.	IC, responders, stakeholders.	Per incident.
Incident bridge (Zoom)	Voice/video war room; screen-share for triage.	IC, responders.	Opened at incident declaration.
Status page (external)	User-visible status; maintenance; incidents.	End users (scholars, librarians, readers).	Updated within 30 min of user-visible impact.
Email: csa-notify@	Notify CSA of SEV-1 within 1 hour.	Chief Systems Architect.	Per SEV-1.
Email: postmortem@	File postmortem within 5 business days of SEV-1/2.	Engineering leadership; CSA.	Per SEV-1/2.

Convention. Every SEV-1 and SEV-2 SHALL have a dedicated #incident-<id> channel, an incident bridge, and an incident record in the incident-management system. The Incident Commander SHALL post a status update to the war-room channel at minimum every 30 minutes during an active incident, even if the update is “still investigating.” Silence breeds panic; visible progress breeds calm (Playbook Ch 16 §16.5).

2.5 The On-Call Rotation

Every production service SHALL be covered by a continuous on-call rotation consisting of at least a primary and a secondary (Playbook Ch 13 §13.2). The rotation SHALL be published eight weeks in advance, SHALL have no gaps, and SHALL have no engineer on PTO or carrying another service's primary concurrently.

2.5.1 Primary On-Call

The primary is the first responder. Acknowledgement SLA is 5 minutes. The primary mitigates, escalates, declares incidents, invokes break-glass within authority, and writes the post-incident timeline. The primary SHALL NOT carry the pager for more than one week (Playbook Ch 13 §13.7).

2.5.2 Secondary On-Call

The secondary is the primary's escalation target and backup. Acknowledgement SLA is 10 minutes. If the primary has not acknowledged within 5 minutes, the secondary becomes the primary. The secondary SHALL be available to join the war room, to research the runbook while the primary mitigates, and to assume the pager when the primary takes a post-incident recovery break.

2.5.3 Handoff at 10:00 UTC

The rotation hands off at 10:00 UTC every Monday. Handoff is the most error-prone moment in the rotation (Playbook Ch 13 §13.6). The outgoing primary SHALL brief the incoming primary using the handoff checklist (Appendix A), SHALL transfer the pager with a test page acknowledgement, and SHALL record the handoff in the rotation log. The handoff SHALL NOT occur until every open item has been carried forward in writing; verbal handoff is forbidden.

2.5.4 Handoff Checklist (Summary)

Handoff Checklist (Full version: Appendix A)

- ☐ **H-01** Open incidents reviewed; each has a current-state summary.
- ☐ **H-02** Pending alerts from last 24h reviewed; each closed or carried forward.
- ☐ **H-03** In-flight deployments identified with stage, observation window, rollback triggers.
- ☐ **H-04** Scheduled jobs in next 24h identified (backups, reconciliation, DOI deposits).
- ☐ **H-05** Break-glass credential inventory confirmed intact and not expired.
- ☐ **H-06** Constitutional Health dashboard confirmed green at handoff moment.
- ☐ **H-07** SLO dashboard burn rates reviewed; any SLO above 2x burn carried forward.
- ☐ **H-08** Test page acknowledged by incoming primary on their device.
- ☐ **H-09** Rotation log entry recorded: timestamp, incoming, outgoing, carried-forward items.
- ☐ **H-10** Anomaly watch list handed off with owner and next-action.

Chapter 3 — Routine Operations

Routine operations are the daily, weekly, monthly, and quarterly procedures that keep the platform healthy. They are the operational form of the Constitutional principle that what is not verified is not trusted: the operator verifies the invariants daily, the SLOs weekly, the traceability graph weekly, the credentials monthly, the dependencies monthly, and the disaster-recovery capability quarterly. Each routine operation is a procedure with a defined cadence, owner, steps, verification, and escalation.

3.1 Daily Constitutional Health Check

When. Every day, before 10:00 UTC, by the outgoing primary during handoff and again by the incoming primary within the first 30 minutes of the shift.

Who. The primary on-call.

Purpose. Confirm that every one of the 47 Constitutional invariants (INV-I through INV-L) is green; that no evidence is stale; that no SLO is burning error budget above the 1x rate. This is the operator's morning vitals.

Procedure: Daily Constitutional Health Check

Step 1. Open the Constitutional Health dashboard grafana.ops.opuspublica.com/d/constitutional-health

Step 2. Verify every invariant panel is green Panels cover INV-I (identity: 13 criteria), INV-A (auditability: 4 criteria), INV-T (traceability: 8 rules), INV-D (durability), INV-S (security), and INV-L (lifecycle). Any amber or red panel SHALL be triaged before handoff completes.

Step 3. Verify no stale evidence Each panel shows the timestamp of the last evidence collection. Any evidence older than 24 hours SHALL be flagged; the operator SHALL run the evidence collection job manually if the staleness exceeds 6 hours.

Step 4. Open the SLO Burn dashboard grafana.ops.opuspublica.com/d/slo-burn

Step 5. Verify no SLO is burning above 1x If any SLO is burning between 1x and 2x, file an SLO-watch ticket. If any SLO is burning above 2x, follow the SLO burn-rate runbook (§5.3). If any SLO is burning above 6x, declare SEV-2 (§6.6).

Step 6. Open the Audit Chain dashboard Confirm INV-A-03 (cryptographic chain) is verifiable end-to-end. The panel shows the result of the last chain-verification job; if the job has not run in the last hour, run it manually:

Step 7. Run the chain-verification job if stale kubectl --context prod-eu -n opus-publica create job --from=cronjob/audit-chain-verify audit-chain-verify-manual-$(date +%s)

Step 8. Record the check in the daily-ops log Post to #ops: "Daily constitutional health check <date>: all green / <anomaly>." The post is the audit artifact that the check was performed.

Verification. The check is verified when the daily-ops log post is created and every panel is confirmed green (or every anomaly is triaged with a ticket number).

Escalation. If any INV-I or INV-A panel is red, the operator SHALL declare SEV-1 immediately and follow §6.3 or §6.4. If INV-T or INV-S panels are red, follow §6.7. If the dashboard itself is unreachable, follow §6.7 (treat as a major-subsystem outage).

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.2 Daily Certification Coverage Check

When. Every day, before 11:00 UTC, by the incoming primary.

Who. The primary on-call.

Purpose. Verify that every GATE criterion in the Certification Checklist (CC-INV-X-NN) has produced a PASS verdict in the last 24 hours. A GATE criterion that has not produced evidence in 24 hours is, by definition, an un-operationalized rule, and SHALL be flagged.

Procedure: Daily Certification Coverage Check

Step 1. **Open Backstage: Scorecards → Certification Coverage**
backstage.opuspublica.com/scorecards/certification-coverage

Step 2. **Verify every GATE criterion shows a PASS within the last 24 hours** The view lists all 47 invariants × their certification criteria (CC-INV-*). Each row shows the last verdict and timestamp.

Step 3. **For any criterion showing STALE (>24h) or FAIL** Click through to the evidence detail. If STALE, run the evidence collection job for that criterion. If FAIL, file a CC-failure ticket and follow the linked runbook.

Step 4. **Verify the Production Acceptance Gate is green** The Gate is the conjunction of all CC-INV-* criteria. If the Gate is red, no release SHALL proceed (§4.2).

Step 5. **Post the coverage summary to #ops** "Certification coverage <date>: 47/47 PASS / <criterion> STALE (ticket #NN)."

Verification. The Backstage view shows zero STALE or FAIL rows, or every exception has an open ticket.

Escalation. If three or more GATE criteria are FAIL or STALE simultaneously, escalate to the Tech Lead; the platform is in a degraded-certification posture and releases SHALL be paused until resolved.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.3 Weekly SLO Review

When. Every Monday at 14:00 UTC, by the SRE team with the primary on-call in attendance.

Who. SRE team lead, primary on-call, Release Manager (optional).

Purpose. Review the previous week's burn rates; decide whether to amend any SLO; decide whether to redirect engineering capacity to error-budget remediation. This is the operational form of the Constitution's SLO management regime (Vol II Ch 32; PB Ch 15 §15.5).

Procedure: Weekly SLO Review

Step 1. **Pull the weekly SLO report** The report is generated by the SLO dashboard: 30-day burn rate per SLO, remaining error budget, projected budget exhaustion date, top contributing incidents.

Step 2. **For each SLO, classify the week** Green (<1x burn), Amber (1–2x burn), Red (>2x burn). Red SLOs trigger the remediation protocol below.

Step 3. **For Red SLOs, decide remediation** Options: (a) shed non-essential load (e.g., defer batch jobs); (b) rollback the most recent release if it correlates; (c) redirect one engineer to error-budget remediation for the week; (d) declare a feature freeze until the SLO returns to Green.

Step 4. **For Amber SLOs, set a watch** Add to the SLO-watch list; re-review at the next daily constitutional health check; if it goes Red, escalate.

Step 5. **Decide SLO amendments** If an SLO has been Amber or Red for 4 consecutive weeks, the SRE team lead SHALL propose an SLO amendment (PB Ch 15 §15.7). Amendments require CSA approval.

Step 6. **Record the review** File the weekly SLO review document in the SLO-review folder; post the summary to #ops.

Verification. The review document is filed; every Red SLO has a remediation ticket; every Amber SLO is on the watch list.

Escalation. If an SLO is projected to exhaust its error budget in less than 6 hours, follow the SEV-2 burn-rate runbook (§6.6). If the SLO amendment process is contentious, escalate to the CSA.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.4 Weekly Traceability Integrity Check

When. Every Friday at 15:00 UTC, by the knowledge-graph team with the primary on-call in attendance.

Who. Knowledge-graph team lead, primary on-call.

Purpose. Run all eight traceability integrity rules (INV-T-01 through INV-T-08) against the full Neo4j graph; remediate any orphans (articles without DOIs, DOIs without articles, audit records without articles, articles without audit records, etc.).

Procedure: Weekly Traceability Integrity Check

Step 1. **Run the full integrity check** kubectl --context prod-eu -n opus-publica create job --from=cronjob/traceability-integrity traceability-integrity-manual-$(date +%s)

Step 2. **Wait for the job to complete (typically 5–20 minutes for the full graph)** kubectl --context prod-eu -n opus-publica wait job/traceability-integrity-manual-<ts> --for=condition=complete --timeout=30m

Step 3. Retrieve the report The job writes a JSON report to the traceability-reports bucket. Pull the report: aws s3 cp s3://opus-traceability-reports/$(date +%Y-%m-%d).json - | jq .

Step 4. Verify every rule passed For each of INV-T-01 through INV-T-08, the report's rules[i].verdict SHALL be PASS. If any rule is FAIL, count the orphan set.

Step 5. For each FAIL rule, remediate the orphans Each rule has a documented remediation playbook in the traceability runbook repository. The most common is INV-T-02 (every article has a DOI): the remediation is to mint the missing DOI via the DOI service and re-reconcile Crossref.

Step 6. Verify the remediation Re-run the integrity check for the affected rule only; confirm the rule now passes and the orphan count is zero.

Step 7. Record the check Post to #ops: "Weekly traceability integrity <date>: 8/8 PASS / <rule> FAIL → remediated (ticket #NN)."

Verification. The integrity-check job completes; the report shows 8/8 PASS; every remediation has a re-verification run.

Escalation. If the integrity-check job fails to complete within 30 minutes, follow the traceability-graph outage runbook (§6.7). If a rule shows >100 orphans, escalate to the Tech Lead; this is a systemic data-integrity event.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.5 Monthly Credential Rotation

When. The first Tuesday of every month, by the platform on-call.

Who. Platform on-call, with Tech Lead approval per the authority matrix (OP-GOV-004, OP-GOV-005).

Purpose. Rotate every external and internal credential per the rotation cadence in PB Ch 14 §14.3. The cadence is:

Credential	Cadence	Owner	Rotation Runbook
Crossref API credentials	30 days	Platform on-call	PB Ch 14 §14.3 — Crossref variant
ORCID API credentials	30 days	Platform on-call	PB Ch 14 §14.3 — ORCID variant
Preservation system API token	30 days	Platform on-call	PB Ch 14 §14.3 — preservation variant
Database admin password (break-glass)	30 days	Platform on-call	PB Ch 14 §14.3 — DB variant
TLS certificate (public endpoints)	90 days (auto-renewed)	Platform on-call (verifier)	cert-manager auto; verify weekly
Neo4j admin password	30 days	Platform on-call	PB Ch 14 §14.3 — Neo4j

			variant
Vault root token	Sealed after use; unsealed only by quorum	CSA + Tech Lead	Quorum-unseal procedure

Procedure: Monthly Credential Rotation (Crossref variant exemplar)

Step 1. **Notify the team** Post to #ops 24 hours in advance: "Credential rotation: Crossref, <date>, 14:00 UTC. Expected downtime: zero. Verify on completion."

Step 2. **Generate the new credential in the Crossref admin console** Crossref admin → Credentials → New credential. Capture the new client ID and secret.

Step 3. **Store the new credential in Vault** vault kv put secret/opus/crossref client_id=<new> client_secret=<new>

Step 4. **Trigger the credential-reload job** kubectl --context prod-eu -n opus-publica rollout restart deployment/crossref-client

Step 5. **Verify the new credential works** Run a test Crossref deposit with a known-good test DOI. The deposit SHALL return 200 OK within 30 seconds.

Step 6. **Revoke the old credential in the Crossref admin console** Crossref admin → Credentials → Revoke <old-credential-id>.

Step 7. **Record the rotation** Post to #ops: "Crossref rotation complete: new credential active, old revoked, test deposit 200 OK." Update the rotation log.

Verification. The test deposit returns 200 OK with the new credential; the old credential returns 401 when invoked; the rotation log shows the new credential's issue date, rotation date, and revocation of the old.

Escalation. If the new credential does not work within 5 minutes of rollout, rollback to the previous credential (still valid until revoked) and contact Crossref support. Do not revoke the old credential until the new is verified.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.6 Monthly Dependency Patching

When. The second Tuesday of every month, by the platform on-call with the owning teams.

Who. Platform on-call coordinates; owning teams execute their own patches.

Purpose. Patch all dependencies per the severity-based cadence in the Engineering Playbook (PB Ch 14 §14.8): Critical 7 days, High 30 days, Medium 90 days, Low as time permits.

Severity (CVSS)	Cadence	Owner	Verification
Critical (9.0–10.0)	7 days from disclosure	Platform on-call + owning	CVE ticket closed; deploys

		team	verified.
High (7.0–8.9)	30 days from disclosure	Owning team	CVE ticket closed; deploys verified.
Medium (4.0–6.9)	90 days from disclosure	Owning team	CVE ticket closed; deploys verified.
Low (<4.0)	As time permits (next release)	Owning team	CVE ticket closed.

Procedure: Monthly Dependency Patching

Step 1. Pull the monthly CVE report The report is generated by the dependency-scanning job: every dependency in every service, with known CVEs, sorted by severity.

Step 2. Triage each Critical CVE within 24 hours For each Critical: identify the affected service, the fix version, and the owning team. Create a Critical-patch ticket with a 7-day deadline.

Step 3. Build the patched version Owning team bumps the dependency, runs CI (including conftest policy verification), and opens a PR.

Step 4. Deploy via the routine-release procedure (§4.2) The patch SHALL go through the normal Production Acceptance Gate; no expedited path is permitted for non-SEV-1 patches.

Step 5. Verify the patch in production Confirm the new version is deployed (kubectl get pods -o jsonpath=...); confirm the CVE scanner no longer reports the CVE.

Step 6. Close the CVE ticket with evidence Attach the scanner report showing the CVE is no longer present; record the deployed version; close the ticket.

Verification. The next dependency-scan report shows zero Critical CVEs open for more than 7 days, zero High CVEs open for more than 30 days, zero Medium CVEs open for more than 90 days.

Escalation. If a Critical CVE cannot be patched within 7 days (e.g., no fix available), the Tech Lead SHALL be notified within 24 hours; a mitigation (WAF rule, network isolation, feature flag off) SHALL be deployed in the interim and recorded in the CVE ticket.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

3.7 Quarterly Disaster Recovery Test

When. The last full week of every quarter, by the DR team with the primary on-call.

Who. DR team lead, primary on-call, Tech Lead, CSA (approver).

Purpose. Verify that the DR plan meets RPO 15 minutes and RTO 1 hour (Vol II Ch 33). The test SHALL exercise the actual failover path (not a simulation), SHALL measure the actual RTO and RPO against the targets, and SHALL produce a test report reviewed by the CSA.

Procedure: Quarterly DR Test

Step 1. Schedule the test Pick a date at least 4 weeks out; notify all teams; freeze releases for the test window (typically 2 hours).

Step 2. Assemble the DR team DR team lead (IC), primary on-call, Tech Lead, CSA (approver on call), comms lead.

Step 3. Declare the DR test (not a real DR activation) Post to #ops and the status page: "Scheduled DR test <date/time>. Expected user impact: brief read-only period."

Step 4. Cut traffic to the primary region Switch DNS to point at the secondary region. Monitor the cutover.

Step 5. Promote the secondary database to primary aws rds promote-read-replica --db-instance-identifier opus-secondary

Step 6. Verify the application is serving traffic from the secondary Run the synthetic-readiness checks: a known-good article fetch, a DOI resolution, a search query. All SHALL return 200 OK.

Step 7. Measure the actual RTO Time from DNS switch to first successful 200 OK. Record against the 1-hour target.

Step 8. Measure the actual RPO Compare the last write in the primary (before switch) to the last replicated write in the secondary. Record against the 15-minute target.

Step 9. Failback to the primary region Reverse the procedure: re-establish replication from secondary back to primary, switch DNS back, verify.

Step 10. Produce the DR test report Document RTO, RPO, gaps, root cause of any gap, corrective actions, owner, deadline. Submit to CSA for review.

Verification. The DR test report shows RTO ≤ 1 hour and RPO ≤ 15 minutes; every gap has a corrective-action ticket.

Escalation. If RTO or RPO exceeds the target by more than 10%, the test SHALL be repeated within 30 days after corrective actions. If RTO exceeds 2 hours or RPO exceeds 1 hour, the CSA SHALL be notified within 24 hours and the platform's DR posture SHALL be re-evaluated.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

Chapter 4 — Deployment Operations

Deployment operations cover the routine release, the rollback, the hotfix, and the feature-flag lifecycle. All four are governed by the Production Acceptance Gate (Vol II Ch 31; PB Ch 11) and the GitOps deployment model (Flux). The operator's role in deployment is to verify the Gate, observe the rollout, decide whether to promote or rollback, and communicate. The operator SHALL NOT deploy a release that has not passed the Gate; the operator SHALL NOT bypass the Gate except via the break-glass procedure (Chapter 7).

4.1 The Deployment Pipeline

The Opus Publica RC2 platform is deployed via Flux GitOps. The pipeline is:

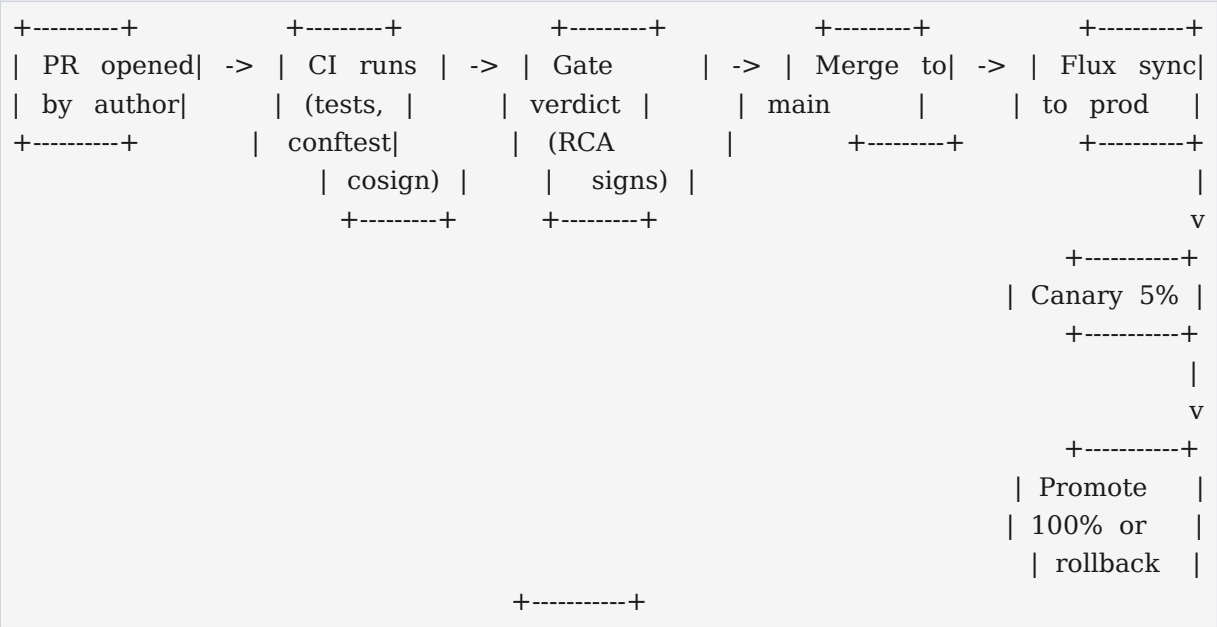

Figure 4.1 — Flux GitOps deployment pipeline (simplified).

What triggers a deploy. A merge to main triggers Flux to reconcile the cluster state with the Git repository. The merge is permitted only after CI has passed (tests, conftest policy, cosign signing) and the RCA has signed the Gate verdict. The operator SHALL NOT merge a PR with a failing Gate verdict; the merge button is disabled by branch protection until the verdict is PASS.

The staged rollout. Flux promotes the change in stages: a 5% canary for 30 minutes, then 25% for 30 minutes, then 100%. At each stage, the operator SHALL observe the SLO dashboard, the error-rate panel, and the latency panel. If any SLO begins to burn at >2x during a canary stage, the operator SHALL rollback immediately (§4.3). The operator SHALL NOT promote to 100% while a canary SLO is burning.

4.2 Deploying a Routine Release

Procedure: Deploy a Routine Release

Step 1. Verify the Gate verdict is PASS Open Backstage: Scorecards → Release → <PR>. The Gate verdict SHALL show PASS, signed by the RCA. If FAIL, do not proceed; the PR author SHALL remediate.

Step 2. Verify the on-call rotation is staffed The No-On-Call-No-Release rule (PB Ch 13 §13.8) applies. If no primary is on the rotation for the affected service, the release SHALL NOT proceed.

Step 3. Merge the PR to main gh pr merge <PR> --squash --delete-branch. The merge triggers CI on main (final conftest, cosign signing of the image, Rekor entry).

Step 4. Observe the CI run gh run watch. The CI run SHALL complete green. If red, the merge SHALL be reverted via git revert and the release re-tried after the failure is diagnosed.

Step 5. Observe Flux sync flux --context prod-eu get kustomization opus-publica. The kustomization SHALL show Ready=True with a recent reconciliation timestamp.

Step 6. Observe the canary stage (5% for 30 minutes) Watch the SLO dashboard. The SLO SHALL remain below 1x burn. Watch the error-rate and latency panels. No new error pattern SHALL appear.

Step 7. Promote to 25% (auto via Flux canary weight) After the 30-minute canary window with no SLO regression, Flux auto-promotes. The operator SHALL observe for another 30 minutes.

Step 8. Promote to 100% Flux auto-promotes. The operator SHALL observe for 30 minutes after full rollout before declaring the release stable.

Step 9. Record the release Post to #ops: "Release <version> deployed: canary 5% → 25% → 100%, SLOs green throughout, observed 30 min post-rollout. RCA: <name>."

Verification. The new version is the only version running in production (kubectl get pods -o jsonpath=...); the SLO dashboard shows no regression for 30 minutes after full rollout; the release record is filed in the release log.

Escalation. If the canary stage shows SLO regression, rollback (§4.3). If Flux fails to sync, follow the CI/CD outage runbook (§6.8). If the new version introduces a user-visible bug not caught in canary, rollback and declare a SEV-2 or SEV-3 incident depending on impact.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

4.3 Rolling Back a Release

Rollback is the operator's most-used deployment action. It SHALL be reflexive, fast, and irreversible once initiated. The operator SHALL NOT attempt to patch-forward during an active SLO regression; rollback first, patch-forward after. This is the operational form of the Constitution's principle that the scholarly record and the user come first; root cause comes after service is restored.

Procedure: Rollback a Release

Step 1. Detect the regression SLO dashboard shows burn >2x; error-rate panel shows new error pattern; user reports come in. The operator SHALL correlate the regression with the deployment time.

Step 2. Decide to rollback The decision is the primary on-call's alone within their authority (OP-GOV-008: On-call approves rollback). If the impact is SEV-1, declare the incident first (§6.3) then rollback; if SEV-2, rollback then declare if needed.

Step 3. Execute the rollback via Flux git revert <merge-commit> → git push. Flux SHALL reconcile back to the previous version. Alternatively: flux --context prod-eu suspend kustomization opus-publica; flux --context prod-eu reconcile kustomization opus-publica --with-source; resume once stable.

Step 4. Verify the rollback Confirm the previous version is running (kubectl get pods -o jsonpath=...). Confirm SLO dashboard returns to <1x burn within 10 minutes. Confirm error rate returns to baseline.

Step 5. Communicate Post to #ops and (if user-visible) the status page: "Rollback to <previous-version> due to <regression>. Service restored. Investigating root cause."

Step 6. Postmortem (if SEV-1 or SEV-2) File a blameless postmortem within 5 business days (Appendix C). The postmortem SHALL identify root cause, contributing factors, and action items to prevent recurrence.

Verification. The previous version is the only version running; the SLO dashboard shows burn <1x for 30 minutes; the rollback record is in the release log; the postmortem (if required) is filed.

Escalation. If the rollback itself fails (Flux cannot reconcile, the previous version is also broken), the operator SHALL declare SEV-1, follow the break-glass deployment procedure (§7.4), and convene the war room. If the rollback succeeds but the SLO does not recover, the regression is not the deploy's fault; follow §6.6 (SLO burn) or the relevant invariant runbook.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

4.4 Deploying a Hotfix

A hotfix is an expedited release to address a SEV-1 or SEV-2 production issue. The hotfix SHALL go through the same Gate as a routine release, but on an expedited timeline (target: 4 hours from branch to production). The hotfix SHALL NOT bypass the Gate; if the Gate cannot be satisfied in time, the break-glass deployment procedure (§7.4) is the alternative.

Procedure: Deploy a Hotfix

Step 1. Branch from main git checkout main && git pull && git checkout -b hotfix/<issue-id>. The branch SHALL be from the current production main, not from a feature branch.

Step 2. Implement the fix Minimal change. The hotfix SHALL touch only the code necessary to address the SEV-1/2; unrelated changes SHALL be deferred to a routine release.

Step 3. Run expedited CI gh pr create —label hotfix. CI runs the same checks as a routine PR (tests, conftest, cosign) but is prioritized in the queue.

Step 4. **RCA + CSA approval (expedited)** The RCA SHALL review and sign the Gate verdict within 30 minutes of CI passing. The CSA SHALL approve the hotfix deployment in writing (Slack confirmation is sufficient; the message SHALL be screenshotted into the release record).

Step 5. **Merge to main and deploy via Flux** Follow the routine-release procedure (§4.2) but skip the 25% canary stage (5% → 100% with a 30-minute observation window at each).

Step 6. **Verify in production** Confirm the SEV-1/2 is mitigated; SLO dashboard recovers; the original symptom is gone.

Step 7. **Postmortem** Every hotfix SHALL have a postmortem, even if the hotfix succeeds. The postmortem asks: why was the hotfix necessary? what routine-release failure allowed the bug to reach production? what test would have caught it?

Verification. The hotfix version is running in production; the SEV-1/2 is mitigated; SLO recovers; the postmortem is filed.

Escalation. If the hotfix does not mitigate the SEV-1/2, rollback to the previous stable version and reconvene the war room. If the Gate cannot be satisfied (e.g., the hotfix breaks a test), the CSA SHALL decide between (a) postponing the hotfix until the test is fixed, or (b) invoking break-glass deployment (§7.4).

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

4.5 Managing Feature Flags

Feature flags decouple deployment from release. The operator SHALL use feature flags for any change that has user-visible impact, any change that the team is not 100% confident in, and any change that may need to be rolled back faster than a Flux rollback allows. The flag lifecycle is: creation, canary, promotion, retirement.

4.5.1 Flag Creation

Procedure: Create a Feature Flag

Step 1. **Define the flag** Name: <service>.<feature>. Default: OFF. Owner: <team>. Description: <what it gates>. Expiry: <date> — every flag SHALL have an expiry; flags without expiry are technical debt.

Step 2. **Create the flag in the flag-control plane** opa-flag create --service=<service> --name=<feature> --default=off --owner=<team> --expiry=<date>

Step 3. **Wire the flag into the code** The code SHALL check the flag at the decision point and SHALL log every evaluation (for audit). The flag SHALL NOT be checked in a hot path (per-request × 10k req/s); cache the evaluation with a 30s TTL.

Step 4. **Verify the flag exists and is OFF** opa-flag get --service=<service> --name=<feature>

4.5.2 Canary (Partial Rollout)

Procedure: Canary a Feature Flag

Step 1. Set the flag to ON for 5% of traffic opa-flag set --service=<service> --name=<feature> --rollout=5%

Step 2. Observe for 30 minutes Watch the SLO dashboard, error-rate, and the flag-evaluation logs. No regression SHALL appear. If any regression, set the flag back to OFF immediately: opa-flag set --service=<service> --name=<feature> --rollout=0%

Step 3. Promote to 25%, observe 30 minutes Same observation. Promote to 100% or rollback to OFF based on observation.

4.5.3 Promotion and Retirement

Promotion. Once a flag has been 100% ON for 30 days with no rollback, the owning team SHALL remove the flag from the code (delete the evaluation, delete the conditional, make the new behaviour unconditional) in the next routine release. The flag SHALL then be retired.

Retirement. opa-flag delete --service=<service> --name=<feature>. The flag SHALL be deleted within 60 days of 100% promotion; flags older than 60 days post-promotion are technical debt and SHALL be tracked in the flag-debt report reviewed at the weekly SLO review.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

Chapter 5 — Monitoring and Alerting

Monitoring and alerting are the operator's nervous system. This chapter defines the alert taxonomy, the runbooks for each alert class, and the discipline required to prevent alert fatigue. The operator SHALL treat every page as a real event until proven otherwise; the operator SHALL also treat every false page as a runbook-quality ticket and SHALL NOT silence an alert without filing one.

5.1 The Alert Taxonomy

Opus Publica uses a five-level severity taxonomy (Playbook Ch 16 §16.3). Severity governs acknowledgement SLA, escalation path, communication requirements, and postmortem obligation.

Severity	Definition	Ack SLA	Escalation	Postmortem?
SEV-1	Constitutional invariant violation; user-visible data loss; audit-chain break; platform-wide outage.	5 min	Primary → Secondary → Tech Lead → CSA within 1 hour.	Mandatory, within 5 business days.
SEV-2	Critical SLO burn (>6x); major-subsystem outage (Neo4j, CI/CD, Crossref deposit); DR required.	5 min	Primary → Secondary → Tech Lead.	Mandatory, within 5 business days.
SEV-3	Degraded service; anomalous behaviour requiring investigation; non-critical SLO burn (2–6x).	15 min	Primary → Secondary.	Optional, at Tech Lead discretion.
SEV-4	Minor degradation; cosmetic; user-internal-only impact.	30 min	Primary only.	Not required; ticket only.
SEV-5	Informational; no action required; logged for trend analysis.	No ack	None.	Not required.

5.2 Invariant Violation Alerts (SEV-1)

SEV-1

An invariant violation is the most severe alert on the platform. It means a Constitutional contract is broken: a duplicate DOI was minted (INV-I-02), the audit chain is unverifiable (INV-A-03), an article is missing its traceable audit record (INV-T-01), or a published PDF hash has drifted (INV-I-03-b). The operator SHALL treat any invariant violation as SEV-1 and SHALL follow the runbook in §6.3 or §6.4 within 5 minutes of acknowledgement.

The first 5 minutes of an invariant-violation page are scripted (Playbook Ch 13 §13.8, the Five-A pattern): Acknowledge, Assess, Act, Alert, Ascertain. The operator SHALL NOT spend the first 5 minutes reading the Constitution; the operator SHALL spend them following the runbook. The Constitution comes after the service is stable.

Runbook. See §6.3 (Runbook: SEV-1 Invariant Violation) and §6.4 (Runbook: SEV-1 Audit Chain Break).

5.3 SLO Burn-Rate Alerts (SEV-2 / SEV-3)

SLO burn-rate alerts fire when an SLO is consuming its error budget faster than allowed. The alerting uses multi-window multi-burn-rate (Google SRE Workbook Ch 5), with two windows: a short window (5 minutes) and a long window (1 hour). The product of burn rates triggers different severities.

Burn pattern	Severity	Action	Runbook
2x × 1h AND 6x × 5m	SEV-3	Investigate; consider rollback; add to SLO-watch.	§6.9 (anomaly) + §4.3 (rollback)
6x × 1h AND 30x × 5m	SEV-2	Rollback now; declare incident.	§6.6
Project exhaustion <6h	SEV-2	Declare incident; convene war room; consider feature freeze.	§6.6

When to rollback. The operator SHALL rollback a release when (a) the SLO burn-rate is >6x for 5 minutes AND the release was deployed in the last 2 hours; or (b) the projected budget exhaustion is <6 hours and the release is the most likely contributor. The operator SHALL NOT wait for certainty; rollback is cheap, investigation is expensive, and the scholarly record is not the place for 'wait and see.'

5.4 Drift Alerts (SEV-2 / SEV-3)

Drift alerts fire when the production cluster state diverges from the Git repository (infra drift) or when a Constitutional invariant that was green becomes amber without an accompanying deploy (constitutional drift). Drift is the canary of unmanaged change.

5.4.1 Infra Drift

Flux continuously reconciles the cluster to Git. If Flux reports drift (cluster state != desired state) for more than 10 minutes, the alert fires. Causes: a manual kubectl apply (forbidden except in break-glass), a controller misbehaving, or a node failure causing state divergence. The operator SHALL investigate, SHALL NOT silence the alert, and SHALL file a drift ticket.

5.4.2 Constitutional Drift

The Constitutional Health dashboard polls each invariant every 5 minutes. If an invariant that was green becomes amber without a deploy in the preceding 2 hours, the alert fires. Causes: a back-end service degraded, an external integration (Crossref, ORCID) slow, an evidence-collection job failed. The operator SHALL treat as SEV-3, investigate, and either remediate or escalate to the SEV-1 invariant-violation runbook if the invariant turns red.

5.5 Self-Audit Findings

The self-audit job runs nightly at 02:00 UTC and produces a report of every anomaly detected: missing evidence, drift, stale credentials, open CVEs, orphan features, SLO-budget projections. The report is posted to #ops and filed in the self-audit log.

Triage. The primary on-call SHALL triage the self-audit report at the start of every shift. Triage assigns each finding to one of: file-and-forget (informational), watch (track for trend), ticket (action within cadence), or escalate (action now). Findings that map to an invariant SHALL be treated as SEV-3 minimum and SHALL be investigated within 24 hours.

5.6 Alert Fatigue Management

Alert fatigue is the operator's silent enemy. An operator who pages 20 times a day becomes numb; the 21st page, the one that matters, is treated like the other 20. The platform SHALL manage alert load by (a) tuning thresholds based on observed false-positive rates, (b) silencing alerts during scheduled maintenance, (c) routing noisy alerts to a daily digest instead of the pager, and (d) holding a monthly alert-quality review.

5.6.1 Tuning Thresholds

Any alert with a false-positive rate >20% over 30 days SHALL be tuned. The SRE team lead SHALL propose the new threshold; the CSA SHALL approve. The tuning SHALL be recorded in the alert registry with the old threshold, the new threshold, the rationale, and the expected impact on false-positive rate.

5.6.2 Silencing During Maintenance

Scheduled maintenance windows SHALL silence the affected alerts in PagerDuty. The silence SHALL be time-boxed (auto-expire), SHALL identify the maintenance window in the silence message, and SHALL be lifted immediately if the maintenance causes user-visible impact. Silences SHALL NOT be open-ended; an open-ended silence is a missing alert.

5.6.3 Monthly Alert-Quality Review

Every month, the SRE team lead SHALL review the alert-registry: every alert that fired in the last 30 days, with its acknowledgement time, its outcome (true positive / false positive / self-resolved), and its current threshold. Alerts with high false-positive rates SHALL be tuned or retired; alerts that never fired SHALL be reviewed for relevance. The review SHALL be filed in the alert-quality log.

Chapter 6 — Incident Response

Incident response is the operator's most-tested skill. This chapter defines the incident lifecycle, the Incident Commander role, and the runbooks for the seven most likely SEV-1 and SEV-2 scenarios and one SEV-3 investigation. Every runbook follows the standard structure: Trigger, Severity, Prerequisites, Steps, Verification, Escalation, Post-Action, Last-Reviewed. The operator SHALL follow the runbook step-by-step; if the runbook does not cover the observed symptom, the operator SHALL escalate to the SME and the runbook SHALL be amended post-incident.

6.1 The Incident Lifecycle

Every incident moves through five phases: Detect, Triage, Mitigate, Resolve, Postmortem. NIST SP 800-61 Rev. 2 describes the lifecycle as four phases (Preparation; Detection and Analysis; Containment, Eradication, and Recovery; Post-Incident Activity); this manual collapses Preparation into the operator's toolkit (Chapter 2) and splits Mitigation from Resolution for operational clarity.

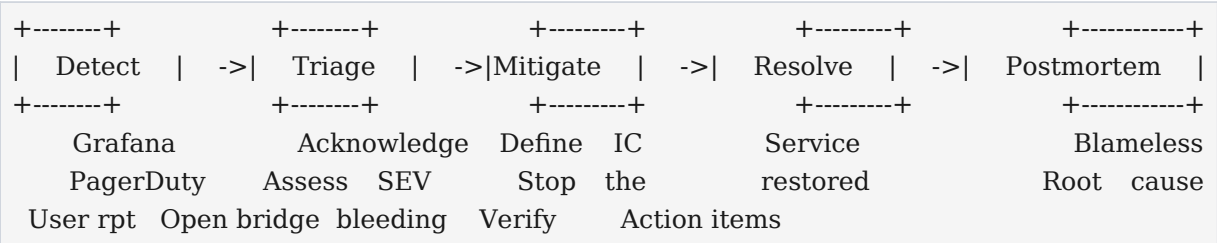

Figure 6.1 — The incident lifecycle.

Phase	Operator Action	Time Target	Output
Detect	Pager fires; operator acknowledges.	5 min ack	Acknowledgement timestamp.
Triage	Read alert payload; open runbook; assess SEV; declare incident.	10 min from ack	SEV declaration; war-room opened.
Mitigate	Execute runbook to stop the bleeding (rollback, freeze, isolate).	30 min from declaration	User-visible impact stopped.
Resolve	Identify root cause; remediate; verify service restored.	Hours (SEV dependent)	Service restored; verified.
Postmortem	Blameless write-up; root cause; action items.	5 business days	Postmortem filed; action items tracked.

6.2 Incident Command

The Incident Commander (IC) is the single human accountable for the incident's coordination. The IC does not mitigate; the IC coordinates. The IC is the operator who declared the incident (typically the primary on-call) until the IC role is explicitly transferred. The IC SHALL be a single person at all times; co-ICs are forbidden because dual command produces contradictory instructions under stress.

6.2.1 The IC Role

- **Convene —** Open and name the war-room channel (#incident-<id>) and the bridge.
- **Communicate —** Establish a 30-minute communication cadence; post an update even if it is only "still investigating."
- **Assign —** Assign responders to specific tasks; do not let everyone chase the same symptom.
- **Decide —** Decide rollback, break-glass, DR activation; the IC owns these decisions and SHALL record them in the timeline.
- **Resolve —** Decide when the incident is resolved; close the war room; stand down responders.
- **Post —** File the postmortem within 5 business days; track action items to closure.

6.2.2 War Room

The war room is the dedicated Slack channel (#incident-<id>) and Zoom bridge. All incident discussion SHALL happen in the war room; side channels fragment information. The IC SHALL pin the incident timeline message at the top of the channel and SHALL update it as the incident progresses. The timeline is the postmortem's primary source.

6.2.3 Communication Cadence

The IC SHALL post to the war-room channel every 30 minutes during an active incident, at minimum. The update SHALL contain: current status (investigating / mitigating / verifying / resolved), what is known, what is unknown, what is being tried, who is doing what, expected next update. For SEV-1, the IC SHALL additionally post to the external status page every 60 minutes during user-visible impact.

6.3 Runbook: SEV-1 Invariant Violation — INV-I-02 (Duplicate DOI)

RB-6.3 — SEV-1 Invariant Violation — Duplicate DOI Minted (INV-I-02)

Trigger	Grafana Constitutional Health panel for INV-I-02 (one DOI per article version) turns red; PagerDuty SEV-1 fires; alert payload includes the offending article IDs and the duplicate DOI.
Severity	SEV-1 — Constitutional invariant violation. The publication pipeline is producing duplicate DOIs, which corrupts the scholarly record's persistent-identifier contract.

Prerequisites	Primary on-call acknowledged the page within 5 min; operator has operator-read role assumed; Crossref admin console accessible; DOI service kubectl access; war-room channel open.
Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge the page Acknowledge in PagerDuty within 5 minutes. Post to #ops: "SEV-1 INV-I-02 duplicate DOI: investigating. War room: #incident-<id>."

Step 2. Declare the incident Open #incident-<id>; assume IC role; open the bridge; pin the timeline message.

Step 3. Confirm the violation Run the invariant check directly: kubectl --context prod-eu -n opus-publica exec deploy/doi-service -- ./check-invariant INV-I-02. Confirm the offending article IDs and the duplicate DOI are reported.

Step 4. Identify scope Query the DOI service for all DOIs minted in the last 24 hours; identify how many articles have duplicates. If >10, this is a mass-duplicate event; mass-mitigation procedure applies.

Step 5. Halt publication (mitigate) Toggle the publish-queue feature flag to OFF: opa-flag set --service=publication --name=publish-queue --rollout=0%. This stops new articles from being published and stops new DOIs from being minted. The scholarly record is now frozen; existing articles remain readable.

Step 6. Freeze deploys flux --context prod-eu suspend kustomization opus-publica. No new deploys SHALL proceed until the violation is resolved.

Step 7. Isolate the affected article(s) For each affected article, mark it as 'under investigation' in the article service: this hides it from search but preserves the canonical record.

Step 8. Merge the duplicate DOI Break-glass is NOT required for this step; the DOI service has a merge-doi command: kubectl --context prod-eu -n opus-publica exec deploy/doi-service -- ./merge-doi --canonical=<canonical-doi> --duplicate=<dup-doi>. The merge SHALL be logged in the audit chain (INV-A-01).

Step 9. Re-reconcile Crossref Run the Crossref reconciliation job for the affected DOIs: kubectl --context prod-eu -n opus-publica create job --from=cronjob/crossref-reconcile crossref-reconcile-manual-$(date +%s). Verify the canonical DOI is the only one registered at Crossref.

Step 10. Verify INV-I-02 restored Re-run the invariant check; confirm INV-I-02 is green. Confirm the Constitutional Health dashboard reflects green within 5 minutes.

Step 11. Unfreeze publication opa-flag set --service=publication --name=publish-queue --rollout=100%. Resume Flux: flux --context prod-eu resume kustomization opus-publica.

Verification

INV-I-02 panel is green for 30 minutes; Crossref reconciliation shows only the canonical DOIs; the publication queue is processing new articles without producing duplicates; the audit chain contains the merge-doi entries; the incident timeline is complete.

Escalation

If the invariant cannot be restored within 1 hour, escalate to the CSA (email csa-notify@). If the root cause is a code bug, the owning team SHALL be paged to prepare a hotfix (§4.4). If the duplicate DOIs were registered at Crossref before merge, Crossref support SHALL be contacted to deprecate the duplicates (may take 24–72 hours; track as a follow-up action item).

Post-Action

Notify CSA within 1 hour of declaration (email csa-notify@ with incident ID, summary, current state). File blameless postmortem within 5 business days (Appendix C). Identify root cause (likely a race condition in the DOI-minting service or a missing uniqueness constraint). Track action items to closure. Update this runbook if the response could have been faster.

6.4 Runbook: SEV-1 Audit Chain Break — INV-A-03

RB-6.4 — SEV-1 Audit Chain Break — INV-A-03 (Cryptographic Chain Verifiable)

Trigger	Grafana Audit Chain dashboard panel for INV-A-03 turns red; the nightly chain-verification job fails; PagerDuty SEV-1 fires. Indicates the audit log has been tampered with, has gaps, or the cryptographic chain is broken.
Severity	SEV-1 — The audit chain is the platform's tamper-evidence guarantee (Vol II Ch 26 INV-A-03). A break means the platform cannot prove that no silent state change occurred. This is the most severe integrity event on the platform.
Prerequisites	Primary on-call acknowledged; operator has operator-read role; audit-log explorer accessible; Vault access to break-glass credential path (if remediation requires direct DB write); war-room open; CSA paged within 5 minutes (this runbook page-escalates to CSA faster than the 1-hour standard).
Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge and declare SEV-1 Acknowledge in PagerDuty within 5 minutes. Post to #ops and open #incident-<id>.

Step 2. Page the CSA immediately INV-A-03 break is one of the two events that require immediate CSA notification (the other is DR activation). Email csa-notify@ AND Slack-mention @csa in #incident-<id>.

Step 3. Confirm the break Retrieve the chain-verification report from the failed job: kubectl --context prod-eu -n opus-publica logs job/audit-chain-verify-<ts>. Identify which block in the chain failed verification and the type of failure (hash mismatch, missing block, timestamp non-monotonic).

Step 4. Freeze the platform Toggle publish-queue OFF; suspend Flux; suspend all cronjobs that write to the audit log: kubectl --context prod-eu -n opus-publica patch cronjob crossref-deposit -p '{"spec": {"suspend":true}}' (repeat for each audit-writing cronjob).

Step 5. Isolate the audit log Mark the audit log read-only at the storage layer (this is a break-glass action; follow §7.3). Capture a forensic snapshot: aws s3 cp s3://opus-audit-log/ s3://opus-forensic-snapshots/<incident-id>/ --recursive.

Step 6. Identify the tampering window Compare the audit log against the secondary-region replica (which is read-only and asynchronous; it may be intact where primary is not). Identify the first divergent record; this is the tampering window.

Step 7. Decide remediation with CSA Three options: (a) restore from the last verified-good backup (acceptable if data loss <15 min, which is within RPO); (b) reconstruct the missing records from the application event log and re-sign the chain (requires CSA approval; long; lossy); (c) accept the gap and re-anchor the chain forward (requires CSA approval; documents the gap permanently). The CSA decides.

Step 8. Execute remediation If option (a): restore the audit log from the backup; verify the chain; resume audit writes. If option (b) or (c): execute the CSA-approved procedure step-by-step; record every action in the incident timeline.

Step 9. Verify INV-A-03 restored Run the chain-verification job manually; confirm green. Confirm the dashboard reflects green within 5 minutes.

Step 10. Unfreeze the platform Resume cronjobs; resume Flux; toggle publish-queue ON only after the CSA confirms the audit chain is trustworthy.

Verification

INV-A-03 panel is green for 30 minutes; the chain-verification job completes successfully; the CSA has confirmed in writing (Slack) that the audit chain is trustworthy; the platform is unfrozen and operating normally.

Escalation

Page the CSA immediately upon declaration; do not wait the standard 1 hour. If the CSA is unreachable within 15 minutes, escalate to the Tech Lead, who SHALL assume CSA authority for this incident only (record in timeline). If remediation cannot be completed within 4 hours, consider DR activation (§8.3).

Post-Action

Notify CSA within 1 hour of declaration (already paged; confirm with email summary). File blameless postmortem within 5 business days. The postmortem SHALL address: how was the tampering introduced? what control failed? what additional monitoring would detect it sooner? should the platform move to a stronger tamper-evidence mechanism (e.g., external notarization)? Engage security team for forensic analysis; preserve the forensic snapshot for at least 1 year.

6.5 Runbook: SEV-1 Crossref Deposit Failure (Mass Deposit)

RB-6.5 — SEV-1 Crossref Mass Deposit Failure — SLO-04 at Risk

Trigger	Crossref deposit SLO (SLO-04: 99.5% of articles deposited within 24 hours of publication) burn-rate >6x for 5 minutes; OR the Crossref deposit queue depth >1000 for >1 hour; OR Crossref returns 5xx for >5% of deposit attempts over 30 minutes.
Severity	SEV-1 — Articles without DOIs registered at Crossref are not persistently citable; the scholarly record's persistence contract (Vol II Ch 26 INV-D-01) is at risk. Mass deposit failure is the platform's most likely SEV-1.
Prerequisites	Primary on-call acknowledged; Crossref admin console accessible; Crossref status page reachable; war-room open.
Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge and declare SEV-1 Acknowledge within 5 minutes; open #incident-<id>; assume IC.

Step 2. Check Crossref status Open https://status.crossref.org. If Crossref is reporting an outage, the runbook is: queue deposits locally, monitor Crossref status, resume deposits when Crossref recovers. Declare the incident but the mitigation is patience.

Step 3. If Crossref status is green, inspect the deposit queue kubectl --context prod-eu -n opus-publica exec deploy/crossref-depositor -- ./queue-status. Identify the failure mode: 4xx (bad metadata) vs 5xx (Crossref problem) vs timeout (network).

Step 4. If 4xx (bad metadata) Identify the offending articles (the queue status shows article IDs and error messages). Quarantine the bad articles: move them to a dead-letter queue so they stop blocking the main queue. The owning team SHALL fix the metadata; this is not a platform-ops task.

Step 5. **If 5xx or timeout** Reduce deposit concurrency to 1 (back off): kubectl --context prod-eu -n opus-publica exec deploy/crossref-depositor -- ./set-concurrency 1. Monitor; if 5xx persists for >30 min, pause deposits entirely (queue locally) and contact Crossref support.

Step 6. **Verify the queue is draining** Watch queue-status every 5 minutes. The queue depth SHALL decrease. If it is not decreasing, escalate to owning team.

Step 7. **Verify SLO-04 recovers** Once the queue is below 100 and Crossref is accepting deposits at normal rate, watch the SLO-04 burn-rate panel. The burn SHALL drop below 2x within 30 minutes.

Step 8. **Communicate to users (if user-visible)** If articles published in the last 24 hours do not yet have resolvable DOIs, post to the status page: "Some articles published in the last 24 hours may not yet resolve via DOI. We are depositing metadata with Crossref and expect resolution within <time>."

Verification

Crossref deposit queue depth <100; SLO-04 burn <1x; the dead-letter queue (if any) has ticketed items; the status-page update (if any) is resolved.

Escalation

If the queue depth continues to grow after 1 hour of mitigation, page the owning team (publication pipeline) to diagnose. If Crossref is the cause and the outage lasts >4 hours, the CSA SHALL be notified (this is approaching the platform's persistence-contract limit).

Post-Action

File blameless postmortem within 5 business days. The postmortem SHALL address: what was the proximate cause? what metadata was bad (if 4xx)? what circuit-breaker would prevent the next occurrence? should the deposit concurrency be auto-tuned based on Crossref 5xx rate? Update this runbook.

6.6 Runbook: SEV-2 SLO Burn Rate Critical

RB-6.6 — SEV-2 SLO Burn Rate Critical — Error Budget Exhausting in <6h

Trigger	Multi-window multi-burn-rate alert: 6x × 1h AND 30x × 5m for any SLO; OR the projected error-budget exhaustion is <6 hours for any SLO.
Severity	SEV-2 — The SLO is at imminent risk of breach; if breached, the platform's service-level commitment to users is broken and the SLO-amendment process (PB Ch 15 §15.7) may be required.
Prerequisites	Primary on-call acknowledged; SLO dashboard accessible; deployment console accessible; war-room open.

Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge and declare SEV-2 Open #incident-<id>; assume IC; post initial update.

Step 2. Identify the burning SLO and the likely cause Open the SLO dashboard. Identify which SLO is burning. Cross-reference with the deployment console: was there a deploy in the last 2 hours? If yes, the deploy is the most likely cause.

Step 3. If a deploy is the likely cause, rollback immediately Follow §4.3 (rollback). Rollback is the default mitigation; do not investigate before rolling back. The scholarly record and the user come first.

Step 4. If no recent deploy, identify the failing component Inspect the SLO's contributing alerts: which SLI is regressing? Which service is the upstream cause? Walk the dependency graph in the traceability dashboard.

Step 5. Mitigate the failing component Options: shed non-essential load (defer batch jobs, throttle low-priority traffic), toggle the relevant feature flag OFF, scale up the failing service (kubectl scale), or fail over to the secondary if the primary is degraded.

Step 6. Verify the burn rate is dropping Watch the SLO dashboard. The burn rate SHALL drop below 2x within 15 minutes of mitigation. If not, escalate.

Step 7. Decide whether to declare a feature freeze If the SLO has been Amber or Red for >2 weeks, the SRE team lead SHALL be paged to consider a feature freeze (PB Ch 15 §15.6) until the SLO returns to Green.

Verification

The burning SLO's burn rate is <1x for 30 minutes; the SLO's projected exhaustion date is >30 days; the mitigation is recorded in the incident timeline.

Escalation

If the burn rate does not drop within 15 minutes of mitigation, page the owning team. If the SLO is actually breached (error budget exhausted), escalate to SEV-1, notify the CSA, and begin the SLO-amendment process.

Post-Action

File blameless postmortem within 5 business days. Update the weekly SLO review (§3.3) with this incident's contribution to the burn. If the SLO was actually breached, the SLO-amendment proposal SHALL be drafted within 10 business days.

6.7 Runbook: SEV-2 Traceability Graph Outage (Neo4j Down)

RB-6.7 — SEV-2 Traceability Graph Outage — Neo4j Down; Integrity Checks Blind

Trigger	Neo4j health probe fails for >2 minutes; OR the

	traceability-integrity cronjob fails to connect; OR the traceability dashboard panel is red for >5 minutes.
Severity	SEV-2 — The traceability graph is the platform's integrity-verification engine. While it is down, INV-T-01 through INV-T-08 cannot be verified; orphans cannot be detected; the platform is flying blind on data integrity.
Prerequisites	Primary on-call acknowledged; Neo4j admin credentials in Vault; war-room open; knowledge-graph team paged.
Last-Reviewed	09 August 2026

Steps

Step 1. **Acknowledge and declare SEV-2** Open #incident-<id>; page the knowledge-graph team as responders; assume IC.

Step 2. **Confirm Neo4j is down** kubectl --context prod-eu -n opus-traceability get pods -l app=neo4j. If pods are CrashLoopBackOff, inspect logs: kubectl logs <pod> --previous.

Step 3. **If pods are crashed, attempt restart** kubectl --context prod-eu -n opus-traceability rollout restart statefulset/neo4j. Watch for recovery; if pods restart successfully, verify the graph is queryable.

Step 4. **If restart fails, fail over to the secondary** Neo4j runs as a primary + replica. Promote the replica: kubectl --context prod-eu -n opus-traceability exec statefulset/neo4j-replica -- ./neo4j-admin dbms set-initial-password <temp> && kubectl --context prod-eu -n opus-traceability patch service neo4j -p '{"spec":{"selector":{"app":"neo4j-replica"}}}'

Step 5. **Verify the graph is queryable** Run a known-good Cypher query: kubectl --context prod-eu -n opus-traceability exec deploy/traceability-api -- ./health. The health check SHALL return green.

Step 6. **Run the integrity check** Re-run the traceability-integrity job manually. Confirm INV-T-01 through INV-T-08 all pass.

Step 7. **If integrity check reveals orphans** These orphans may have accumulated while Neo4j was down. Triage per §3.4; remediate in the next 24 hours.

Verification

Neo4j is healthy (green dashboard); the traceability-integrity job passes all 8 rules; the Constitutional Health dashboard shows INV-T-* green.

Escalation

If Neo4j cannot be restored within 1 hour, escalate to SEV-1 (the platform's integrity-verification gap exceeds the Constitutional tolerance). The CSA SHALL be notified. If the data is corrupted, the DR runbook (§8.3) SHALL be considered.

Post-Action

File blameless postmortem within 5 business days. The postmortem SHALL address: why did Neo4j fail? is the replica promotion procedure reliable? should the platform move to a managed Neo4j service? Update this runbook.

6.8 Runbook: SEV-2 CI/CD Outage (GitHub Actions Down)

RB-6.8 — SEV-2 CI/CD Outage — GitHub Actions Down; Releases Blocked

Trigger	GitHub Actions workflow run failure rate >50% over 30 minutes; OR GitHub status page reports Actions degraded; OR Flux sync fails for >10 minutes because the source (Git) is unreachable.
Severity	SEV-2 — The CI/CD pipeline is the platform's change-management mechanism. While it is down, no routine releases, no hotfixes, no rollbacks via Flux can proceed. The platform is in change-frozen state.
Prerequisites	Primary on-call acknowledged; GitHub status page reachable; release-engineering team paged; war-room open.
Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge and declare SEV-2 Open #incident-<id>; page release-engineering as responders; assume IC. Declare an internal change freeze: no merges to main until CI/CD is restored.

Step 2. Confirm the outage scope Check https://www.githubstatus.com. If GitHub Actions is globally degraded, the mitigation is to wait. If only Opus Publica's workflows are failing, diagnose locally.

Step 3. If global GitHub outage Post to #ops: "CI/CD outage: GitHub Actions globally degraded. Change freeze in effect. Hotfixes (if needed) via break-glass deployment (§7.4). Monitoring GitHub status."

Step 4. If local failure, inspect the workflow run gh run view <run-id> --log-failed. Identify the failing step: is it a runner issue, a missing secret, a flaky test, or a policy check failure?

Step 5. If a flaky test is blocking Re-run the workflow: gh run rerun <run-id>. If the flake persists, disable the test (with a ticket) and proceed; the test SHALL be fixed within 24 hours.

Step 6. If a runner issue GitHub-hosted runners cannot be fixed by the operator; self-hosted runners can be restarted: kubectl --context prod-eu -n actions-runner rollout restart deployment/actions-runner

Step 7. Verify CI/CD restored Re-run a known-good workflow; confirm green. Lift the change freeze; communicate to #ops.

Verification

CI/CD workflows are completing green; Flux is syncing normally; the change freeze is lifted; the release-engineering team has confirmed restoration.

Escalation

If CI/CD is down for >2 hours AND a SEV-1 is in progress that requires a hotfix, the break-glass deployment procedure (§7.4) SHALL be invoked with CSA approval. If the outage is global and lasts >4 hours, notify the CSA (the platform's ability to respond to a SEV-1 with a hotfix is compromised).

Post-Action

File blameless postmortem within 5 business days. The postmortem SHALL address: was the outage global or local? what was the longest path to recovery? should the platform maintain a secondary CI/CD system (e.g., a self-hosted GitLab runner as fallback)? Update this runbook.

6.9 Runbook: SEV-3 Opaque Anomaly (ML-Flagged Audit Anomaly)

RB-6.9 — SEV-3 Opaque Anomaly — ML Audit-Log Anomaly; Investigate

Trigger	The anomaly-detection job flags an audit-log pattern as anomalous (unusual time-of-day pattern, unusual actor, unusual operation sequence). The anomaly is not a confirmed invariant violation; it is a leading indicator that warrants investigation.
Severity	SEV-3 — No confirmed impact; investigation required. The operator SHALL treat as SEV-3 until the anomaly is explained; if the explanation reveals an invariant violation, escalate to SEV-1 (RB-6.3 or RB-6.4).
Prerequisites	Primary on-call acknowledged; audit-log explorer accessible; anomaly-detection dashboard accessible.
Last-Reviewed	09 August 2026

Steps

Step 1. Acknowledge the page Acknowledge within 15 minutes (SEV-3 SLA). Post to #ops: "SEV-3 audit anomaly: investigating."

Step 2. **Open the anomaly-detection dashboard** grafana.ops.opuspublica.com/d/anomaly. Identify the flagged anomaly: which records, what pattern, what confidence score.

Step 3. **Pull the audit records** Use the audit-log explorer to pull the flagged records and their context (the 10 records before and after).

Step 4. **Assess for invariant violation** Do the flagged records indicate an INV-A-01 (missing audit record), INV-A-02 (incomplete record), INV-A-03 (chain tamper), or INV-A-04 (silent state change)? If yes, escalate immediately to RB-6.4.

Step 5. **If no invariant violation, identify the benign explanation** Most anomalies have benign explanations: an unusual deploy, an after-hours break-glass that was authorized but unusual, a new background job. Identify the explanation.

Step 6. **If no benign explanation, escalate** Page the security team. The anomaly SHALL be treated as a potential security incident until explained.

Step 7. **Record the investigation** File an anomaly-investigation ticket with: the flagged records, the investigation steps, the explanation (or 'no explanation found; security paged'), and the disposition (closed, escalated, watch).

Verification

The anomaly is either explained (benign) and the ticket closed, or escalated to security with a handoff record. The anomaly-detection dashboard no longer flags the same pattern.

Escalation

Escalate to SEV-1 (RB-6.4) if the anomaly reveals an audit-chain invariant violation. Escalate to the security team if no benign explanation is found within 4 hours. Escalate to the Tech Lead if three or more anomalies of the same type fire in 24 hours (model drift or new attack pattern).

Post-Action

If the anomaly was benign, update the anomaly-detection model's training data to reduce false positives for this pattern. If the anomaly was a real integrity event, file a blameless postmortem and follow the SEV-1 post-action. Update this runbook.

Chapter 7 — Break-Glass Procedures

Break-glass is the operator's last-resort access. It bypasses the Production Acceptance Gate, the application's normal access controls, and the GitOps deployment model. Because break-glass bypasses the very controls that keep the platform safe, it is governed by the strictest procedural regime on the platform (Playbook Ch 14 §14.5; Vol II Ch 34 OP-GOV-011). Every break-glass invocation is time-boxed, audit-logged, and reviewed by the Tech Lead within 24 hours.

7.1 When Break-Glass Is Justified

Break-glass is justified in exactly two situations. Outside these situations, break-glass is forbidden; the operator SHALL use the normal procedure even if it is slower.

Justification 1 — SEV-1 with no other path

Break-glass is justified when a SEV-1 incident is in progress AND the mitigation requires direct production access that the operator-read or operator-deploy role cannot provide. Example: the audit chain is broken (RB-6.4) and remediation requires a direct DB write to restore the chain. The normal Flux-based deploy path is too slow or is itself broken.

Justification 2 — CI/CD outage during incident

Break-glass is justified when the CI/CD pipeline is down (RB-6.8) AND a release or hotfix is required to mitigate an active incident. The operator cannot deploy via Flux; break-glass deployment (§7.4) is the only path.

Break-glass SHALL NOT be used for routine operations, for convenience, for 'just this once' speed, or because the normal procedure is inconvenient. A break-glass invocation that is later judged unjustified is itself a SEV-3 incident; the operator SHALL receive coaching, and the procedure that drove them to break-glass SHALL be improved.

7.2 Break-Glass Production Access

Procedure: Break-Glass Production Access

Step 1. Declare the break-glass intent The on-call declares in #incident-<id>: "Break-glass invoked: <justification>. Requesting CSA approval." The declaration is the audit record that the operator believes break-glass is justified; it is judged post-hoc by the Tech Lead in the break-glass review.

Step 2. Obtain CSA approval The CSA SHALL approve in writing (Slack is sufficient; the message SHALL be screenshotted into the incident timeline). If the CSA is unreachable within 10 minutes, the Tech Lead SHALL assume CSA authority for this invocation only (record in timeline).

Step 3. **Check out the time-boxed credential** vault kv get -field=break_glass_credential secret/opus/break-glass/<service> — the credential is valid for 30 minutes only. The checkout is logged in the audit chain (INV-A-01).

Step 4. **Use the credential for the declared purpose only** The operator SHALL use the credential only for the declared mitigation. Any unrelated action (e.g., reading unrelated data) SHALL be flagged in the break-glass review. The operator SHALL NOT share the credential; the operator SHALL NOT paste the credential into any non-Vault location.

Step 5. **Check the credential back in** vault kv put secret/opus/break-glass/<service> break_glass_credential=<rotated-value> — the credential is rotated after every use. The rotation is logged.

Step 6. **File the break-glass review ticket** Every break-glass invocation SHALL be reviewed by the Tech Lead within 24 hours. The ticket SHALL contain: the declaration, the CSA approval, the credential checkout time, the actions taken, the credential check-in time, and the post-hoc judgment (justified / unjustified).

Step 7. **Record the invocation in the break-glass log** The invocation is appended to the break-glass log: declaration timestamp, CSA approver, operator, justification, actions, check-in timestamp, review ticket ID, reviewer, judgment.

Verification. The credential was checked out, used for the declared purpose only, rotated on check-in, and the review ticket is filed with all required fields.

Escalation. If the operator cannot check the credential back in (e.g., Vault is unreachable), the operator SHALL notify the Tech Lead immediately; the credential SHALL be considered compromised and SHALL be force-rotated by the security team within 1 hour.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

7.3 Break-Glass Database Access

Break-glass database access is read-only on the replica by default. Direct writes to the production primary database are forbidden except via the break-glass deployment procedure (§7.4) and only for the audit-chain-restoration case in RB-6.4.

Procedure: Break-Glass Database Access (Read-Only on Replica)

Step 1. **Declare and obtain CSA approval** Same as §7.2 steps 1–2.

Step 2. **Check out the read-only replica credential** vault kv get -field=replica_read_only secret/opus/db/break-glass — valid 30 minutes; SQL logged at the database layer.

Step 3. **Connect to the replica (never the primary)** psql --host=replica.opus-db —user=<break-glass-user> —dbname=opus. The replica is read-only; any write attempt SHALL fail and SHALL be logged.

Step 4. **Run only SELECT queries** Every SQL statement is logged. The operator SHALL NOT run DDL, DML, or administrative commands. The operator SHALL scope queries to the minimum necessary (limit rows; filter by primary key).

Step 5. **Check the credential back in and file the review ticket** Same as §7.2 steps 6–7.

Verification. The credential was checked out, used read-only on the replica only, every SQL statement is in the database log, and the review ticket is filed.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

7.4 Break-Glass Deployment

Break-glass deployment is the only path to deploy a change to production without the Production Acceptance Gate. It SHALL be used only when the CI/CD pipeline is down (RB-6.8) and a hotfix is required to mitigate an active SEV-1, or when the Gate itself cannot be satisfied in time for a SEV-1 mitigation and the CSA has approved bypassing the Gate.

Procedure: Break-Glass Deployment

Step 1. **Declare and obtain CSA + RCA approval** Both the CSA and the RCA SHALL approve in writing. The RCA's approval acknowledges that the Gate is being bypassed and the release is uncertified. The CSA's approval acknowledges the constitutional implications.

Step 2. **Build the image manually** Build the image from the hotfix branch: docker build -t opus/<service>:<hotfix-version> . Sign with cosign: cosign sign --key <prod-key> opus/<service>:<hotfix-version>. Push to the registry.

Step 3. **Apply the manifest directly to the cluster (bypassing Flux)** kubectl --context prod-eu apply -f manifest-hotfix.yaml. Flux SHALL be suspended to prevent it from reverting the change: flux --context prod-eu suspend kustomization opus-publica.

Step 4. **Verify the deployment** kubectl --context prod-eu -n opus-publica get pods -l app=<service>. Confirm the new version is running. Confirm the SEV-1 is mitigated.

Step 5. **Resume Flux with the hotfix committed to Git** Commit the hotfix manifest to Git: git add manifest-hotfix.yaml && git commit -m 'hotfix: <issue-id>' && git push. Resume Flux: flux --context prod-eu resume kustomization opus-publica. Flux SHALL reconcile without reverting because Git now matches the cluster.

Step 6. **File the break-glass review AND the postmortem** Break-glass deployment triggers both a break-glass review ticket (§7.2 step 6) AND a blameless postmortem (Appendix C) within 5 business days. The postmortem SHALL address: why was the CI/CD pipeline unavailable? what is the fix to prevent recurrence? should the platform maintain a secondary CI/CD fallback?

Verification. The hotfix version is running in production; the SEV-1 is mitigated; Flux is resumed and reconciling; Git matches the cluster; the break-glass review and postmortem are filed.

Escalation. If the hotfix does not mitigate the SEV-1, the operator SHALL revert to the previous version (kubectl apply the previous manifest) and reconvene the war room. If the hotfix introduces a new SEV-1, declare and follow the relevant runbook.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

Chapter 8 — Disaster Recovery

Disaster recovery is the platform's last line of defence: when the primary region is unrecoverable, the operator fails over to the warm standby in the secondary region. DR is governed by Vol II Ch 33 (RPO 15 min, RTO 1 hour) and operationalized in PB Ch 14 §14.6 and this chapter. DR activation is the single most consequential action on the platform; it is authorized only by the CSA.

8.1 The DR Plan

The DR plan is warm standby in a secondary region. The secondary region runs a full copy of the platform: application services (auto-scaled to zero, ready to scale), a read-replica of the production database (kept within 15 minutes of primary via asynchronous replication), a Neo4j replica, an audit-log replica (write-once, read-only), and the Crossref/ORCID/preservation integrations configured but idle. The DNS is the failover mechanism: in DR, DNS is repointed from the primary to the secondary region.

Component	Primary	Secondary (DR)	Replication
Application services	Active (prod-eu).	Warm (zero replicas, ready to scale).	n/a (stateless).
Database (Postgres)	Primary (prod-eu).	Read-replica (prod-us-west).	Asynchronous; ≤15 min lag (RPO).
Audit log (S3 + chain)	Primary (prod-eu).	Read-only replica (prod-us-west).	Continuous; cross-region replication.
Neo4j (traceability)	Primary.	Replica.	Causal cluster; ≤30 s lag.
Object storage (PDFs, JATS)	Primary (prod-eu).	Replicated (prod-us-west).	Cross-region replication; ≤5 min lag.
DNS	eu.opuspublica.com → prod-eu.	Switchable to prod-us-west via Route 53.	TTL 60s.
Crossref/ORCID/ preservation	Active in primary.	Configured in secondary; idle.	Same credentials (rotated monthly).

8.2 DR Activation Authority

DR Activation Authority — CSA Only

Disaster Recovery SHALL be activated only by the Chief Systems Architect (CSA), in writing, with a recorded justification. The on-call SHALL recommend DR activation; the CSA SHALL authorize it. There is no delegation. If the CSA is unreachable, the Tech Lead SHALL assume CSA authority for the DR decision only (record in incident timeline).

The declaration procedure is:

1. The on-call declares the incident SEV-1.
2. The on-call assesses that the primary region is unrecoverable within the RTO (1 hour) and recommends DR activation in #incident-<id>.
3. The on-call pages the CSA via PagerDuty and Slack-mention @csa.
4. The CSA reviews the recommendation, asks clarifying questions, and either authorizes or denies DR activation in writing in #incident-<id>.
5. If authorized, the on-call proceeds to the DR runbook (§8.3). If denied, the on-call continues with in-region recovery.

8.3 DR Runbook

RB-8.3 — Disaster Recovery Activation — Failover to Secondary Region

Trigger	Primary region is unrecoverable within RTO (1 hour); the CSA has authorized DR activation in writing in #incident-<id>.
Severity	SEV-1 (always; DR activation is never for lesser severity).
Prerequisites	CSA authorization in writing; war-room open; DR team assembled (DR team lead as IC, primary on-call, Tech Lead, CSA on call, comms lead).
Last-Reviewed	09 August 2026

Steps

Step 1. Declare DR activation Post to #incident-<id>: "DR ACTIVATED. CSA authorization: <quote>. IC: <name>. Commencing failover." Post to the status page: "Opus Publica is failing over to the secondary region. Expected downtime: <time>."

Step 2. Assemble the DR team Confirm DR team lead, primary on-call, Tech Lead, CSA on call, comms lead are present in the war room. Assign roles: IC, failover operator, verifier, comms.

Step 3. Verify the secondary is ready Confirm the database replica is within 15 minutes of primary: aws rds describe-db-instances --db-instance-identifier opus-secondary | jq '.DBInstances[0].ReadReplicaSourceDBInstanceIdentifier'. Confirm object storage replication is current. Confirm the audit-log replica is current.

Step 4. Stop writes to the primary (if still accepting) If the primary is partially functional, stop the publication queue and suspend Flux. The goal is to ensure no new writes are made that the replica does not have.

Step 5. Promote the secondary database to primary aws rds promote-read-replica --db-instance-identifier opus-secondary. Wait for promotion to complete (typically 2–5 minutes).

Step 6. Fail over DNS aws route53 change-resource-record-sets --hosted-zone-id <zone> --change-batch file://dns-failover.json. The change points opuspublica.com at the secondary region's load balancer. TTL is 60s; expect global propagation within 5 minutes.

Step 7. Scale the secondary application to production capacity kubectl --context prod-us-west -n opus-publica scale deployment/<service> --replicas=<prod>. Repeat for each service.

Step 8. Verify end-to-end Run the synthetic-readiness checks from the secondary: a known-good article fetch, a DOI resolution, a search query, an audit-chain verification. All SHALL return 200 OK / green.

Step 9. Verify the SLO dashboard reflects the secondary The SLO dashboard SHALL now be reading from the secondary region. Confirm SLIs are reporting.

Step 10. Communicate Post to the status page: "Opus Publica is now serving from the secondary region. Service restored. Investigating root cause of primary failure."

Step 11. Stand down the war room when stable The IC SHALL confirm 30 minutes of stable operation before standing down. The war room remains open for the duration of the secondary-region operation; it closes only after failback to the primary.

Verification

The secondary region is serving all traffic; the SLO dashboard is green; the synthetic checks pass for 30 minutes; the status page is updated; the DR activation record is filed in the audit chain (INV-A-01).

Escalation

If the secondary database promotion fails, the DR team SHALL fall back to the most recent verified-good backup (RPO potentially exceeded; CSA SHALL be notified). If DNS failover fails, the DNS team SHALL be paged immediately. If the synthetic checks fail, the war room continues; the IC SHALL consider whether to fail back to primary (if recovered) or continue diagnosing the secondary.

Post-Action

Notify CSA within 1 hour of activation (already paged; confirm with email summary). File blameless postmortem within 5 business days. The postmortem SHALL address: why was DR activated? what was the actual RTO and RPO? what gaps in the DR plan were revealed? what is the failback plan and timeline? Engage the DR team to plan and execute failback to the primary region (typically within 7 days). Track all action items to closure.

8.4 DR Testing

DR SHALL be tested quarterly per §3.7. The test SHALL exercise the actual failover path (not a simulation), SHALL measure actual RTO and RPO against targets, and SHALL produce a test report reviewed by the CSA. A DR plan that is not tested is a DR plan that does not exist; the first real DR activation SHALL NOT be the first time the operator runs the runbook.

What to verify. (1) The secondary database can be promoted within the RTO; (2) DNS failover propagates within 5 minutes; (3) The application scales to production capacity within 10 minutes; (4) The synthetic checks pass; (5) The audit-log replica is current; (6) The failback procedure works; (7) The actual RPO is ≤ 15 minutes; (8) The actual RTO is ≤ 1 hour. Any gap SHALL be tracked as a corrective-action ticket with an owner and a deadline.

8.5 The Legal-Removal Procedure

Legal Removal — The Only Path to Deleting a Published Artifact

A published artifact (article, DOI, PDF, JATS) SHALL NOT be deleted except via the legal-removal procedure (PB Ch 14 §14.7; Vol II Ch 34 OP-GOV-002). Legal removal is authorized only by the Editorial Board + CSA, in response to a court order or a retraction. Direct deletion by any operator is forbidden absolutely; the audit chain SHALL record every legal removal with the authorization reference.

The procedure is:

6. The Editorial Board issues a retraction or receives a court order.
7. The Editorial Board + CSA approve the legal removal in writing.
8. The legal-removal job is run: it marks the artifact as retracted (not deleted), updates the DOI metadata at Crossref to point to a retraction notice, preserves the original artifact in the long-term preservation system (LOCKSS/CLOCKSS), and records the removal in the audit chain with the authorization reference.
9. The operator SHALL NOT run the legal-removal job directly; the Editorial Board SHALL trigger it via the editorial workflow, and the operator SHALL only verify its completion.

Verification. The artifact is marked retracted (not deleted); the DOI resolves to the retraction notice; the original is preserved; the audit chain records the authorization. The artifact is recoverable if the legal-removal decision is reversed.

Last-Reviewed: 09 August 2026 · Owner: Platform Operations · Review cadence: quarterly

Chapter 9 — Operator Health and Wellbeing

On-call is a load. The platform acknowledges this and establishes policy to prevent the load from becoming corrosive (Playbook Ch 13 §13.7). Burned-out operators miss alerts, make poor decisions under stress, and leave the rotation. The platform cannot afford any of these; the policies in this chapter are the operational form of the Constitution's duty to the engineer as a person.

9.1 On-Call Rotation Limits

- **Max consecutive primary shifts —** no engineer SHALL carry the primary pager for more than one week. A two-week primary stint is permitted only when the rotation cannot be staffed otherwise and SHALL be followed by at least one full week off the rotation.
- **No on-call during PTO —** an engineer on PTO SHALL NOT be on the rotation. The Tech Lead SHALL re-roster the rotation before the PTO begins. If the engineer is the only eligible primary, the service's release SHALL be deferred until the engineer returns or a new primary is trained (No-On-Call-No-Release rule, PB Ch 13 §13.8).
- **No concurrent rotations —** an engineer on call for one service SHALL NOT simultaneously be on call for another service. Cross-service concurrency produces divided attention and SHALL be resolved by the Tech Lead before the rotation begins.
- **Night-page recovery —** an engineer who has been paged between 22:00 and 06:00 local for more than 30 minutes of work SHALL be excused from the next morning's standup and MAY take a half-day recovery with no loss of pay or PTO balance.

9.2 Post-Incident Recovery (Cooldown Policy)

After a SEV-1 incident, the primary on-call SHALL be placed on a cooldown: no primary on-call for the next 7 days, no new releases to the affected service for 48 hours without the engineer's explicit consent. The cooldown exists because the operator who just resolved a SEV-1 is fatigued, possibly traumatized, and likely to make a poor decision if paged again immediately. The cooldown is not a punishment; it is a recovery period.

After a SEV-2 incident, the cooldown is 48 hours of no primary on-call. After three SEV-2 incidents in one shift, the cooldown is 7 days (the shift was clearly a high-load one and the operator needs recovery equivalent to a SEV-1).

Where local employment law or collective agreement requires compensation for on-call duty and for pages answered outside working hours, that compensation SHALL be paid. Where no such requirement exists, the organization SHALL nonetheless recognize the load through time-off-in-lieu or equivalent, because unrecognized on-call load produces burnout and burnout produces incidents.

9.3 Escalation and the “Page Up” Culture

The Page-Up Culture

There is no penalty for escalating. There is no penalty for paging the secondary, the Tech Lead, the SME, or the CSA. There IS a penalty for NOT escalating when escalation was warranted: an operator who tries to resolve a SEV-1 alone, fails, and only then escalates has cost the platform time it cannot afford. The operator SHALL page up at the first doubt, not the last resort.

When to escalate:

- **Runbook gap —** the runbook does not cover the observed symptom (page the SME).
- **Authority —** the mitigation exceeds the operator's authority (page the Tech Lead; for break-glass, page the CSA).
- **SEV-1 —** the incident is SEV-1 (page the CSA within 1 hour).
- **Fatigue —** the operator has been working for >2 hours without progress (page the secondary to take over).
- **Uncertainty —** the operator is unsure (page the secondary; uncertainty is sufficient justification).

The page-up culture is reinforced by the blameless postmortem (§9.4): no postmortem SHALL criticize an operator for escalating. A postmortem MAY criticize an operator for NOT escalating.

9.4 The Blameless Postmortem Culture

Every SEV-1 and SEV-2 SHALL have a blameless postmortem (Playbook Ch 16 §16.7; Appendix C). Blameless means: the postmortem SHALL focus on the system, the process, and the runbook, not on the individual. The postmortem SHALL NOT name individuals as causes; it SHALL name behaviours, decisions, and the systemic conditions that produced them. The postmortem SHALL ask 'why was the system vulnerable to this mistake?' not 'who made the mistake?'

Psychological safety is the precondition of blamelessness. An operator who fears punishment will hide mistakes; an operator who hides mistakes produces a platform whose failures are invisible until they are catastrophic. The platform prefers visible small failures to invisible large ones; the blameless postmortem is the mechanism that keeps failures visible.

Learning orientation. The postmortem SHALL produce action items, each with an owner and a deadline. The action items SHALL be tracked to closure at the monthly operations review. A postmortem whose action items are not closed within 90 days is itself a SEV-3 incident: the platform has failed to learn from its own failure.

Chapter 10 — Counterarguments and Limitations

This manual is not a substitute for judgement. It is a compression of the Constitution and the Playbook into a runbook-first reference, and every compression loses information. The operator SHALL use this manual as the first source, not the only source; the operator SHALL escalate to the Playbook and the Constitution when the manual is silent or ambiguous; the operator SHALL escalate to the SME and the CSA when judgement is required. The limitations below are not disclaimers; they are the boundaries within which the manual is authoritative.

10.1 Limitation: Runbooks Age; the Manual Must Be Maintained

A runbook that describes a version of the system that no longer exists is worse than no runbook: it actively misleads the operator at 03:00. The platform evolves — services are renamed, commands change, dashboards are reorganized, invariants are amended — and the manual SHALL evolve with it. The manual SHALL be reviewed quarterly (the Last-Reviewed date on every runbook is the audit artifact). A runbook whose Last-Reviewed date is more than 6 months old SHALL be flagged at the monthly operations review; the owning team SHALL review and either re-affirm or amend.

The maintenance burden is real and the platform accepts it. The alternative — a stale manual — is worse than no manual because it creates false confidence. The operator SHALL treat a runbook step that does not work as a runbook-quality ticket and SHALL NOT silently work around it; the runbook SHALL be amended before the next incident.

10.2 Limitation: No Manual Can Cover Every Scenario; Judgement Is Required

This manual covers the seven most likely SEV-1/2 scenarios and one SEV-3 investigation. It does not cover every scenario. The operator WILL encounter incidents that no runbook anticipates: a novel failure mode, an unusual combination of symptoms, an external dependency behaving in a way nobody has seen. In those moments, the operator's judgement is the platform's last defence.

Judgement is not improvisation. Judgement is the disciplined application of the Five-A pattern (Acknowledge, Assess, Act, Alert, Ascertain) to a novel situation: acknowledge the page, assess the impact, act on the best available mitigation (typically: freeze, rollback, isolate), alert the SME and the CSA, ascertain what happened. The runbook is the script; judgement is the operator's ability to deviate from the script when the script does not fit, while remaining within the authority matrix and the Constitution's invariants.

10.3 Limitation: The Operator's Knowledge Is Tacit; the Manual Captures Only the Explicit

Much of what an experienced operator knows is tacit: the feel of a system that is 'about to' fail, the recognition of a symptom pattern from a previous incident, the intuition that a particular mitigation will

not work in this context. This manual captures the explicit — the commands, the procedures, the verification steps — but it cannot capture the tacit. The tacit is built through shadow shifts (PB Ch 13 §13.3), through postmortem reading, through the war-room experience of watching a senior operator triage.

The implication is that the manual is necessary but not sufficient. An operator who has read the manual but never run a shadow shift is not ready for the pager. The rotation SHALL require both: explicit knowledge (this manual) and tacit knowledge (shadow shifts, postmortems, war-room experience). The shadow-shift requirement is the operational acknowledgement that the manual has limits.

10.4 Counterargument: “Runbooks Are Overhead” — Answered

The argument: writing and maintaining runbooks is engineering time that could be spent on features; runbooks age and become stale; the team knows the system and can respond without written runbooks; the runbook overhead is bureaucratic.

The answer: the cost of a single SEV-1 mishandled for 30 minutes exceeds the cost of every runbook in this manual. The scholarly record's persistence contract is not a feature that can be deferred; an operator without a runbook at 03:00 is an operator who is improvising under stress, and improvisation under stress produces mistakes. Google's SRE Book (sre.google/sre-book/postmortem-culture) is explicit: 'hope is not a strategy.' Runbooks are the strategy; hope is what the operator is left with when the runbook is missing.

The maintenance burden (§10.1) is real but bounded: quarterly review, post-incident amendment, and a runbook-quality ticket for every step that does not work. The team that maintains the runbook is the team that benefits from it; the cost is internalized.

10.5 Counterargument: “Just Hire More SREs” — Answered

The argument: instead of writing runbooks, hire more SREs who know the system deeply and can respond to any incident from their own knowledge.

The answer: more SREs help, but they do not replace runbooks. (1) SREs sleep, take PTO, and change jobs; the runbook does not. A rotation of 10 SREs still has gaps; the runbook has none. (2) Tacit knowledge (§10.3) is built through experience, but the path from new hire to 'knows the system deeply' is 6–12 months; the runbook is the bridge that gets the new hire to their first solo shift. (3) The platform's worst incident is the one that hits when the most experienced SRE is on PTO; the runbook is the institutional memory that survives personnel turnover. (4) The SRE Book (sre.google/sre-book/being-on-call) is explicit that on-call SHOULD be a small fraction of an SRE's time, not the whole of it; runbooks are what make that possible by reducing the cognitive load per page.

The platform's actual answer is: hire enough SREs to staff the rotation sustainably (no engineer on primary for more than one week, no concurrent rotations), AND maintain this manual as the institutional memory that survives turnover. The two are complementary, not alternatives.

Chapter 11 — Conclusion and Implications

11.1 Summary

This manual is the operator's survival guide. It compresses the operational kernel of the Constitution and the Playbook into a runbook-first reference that the operator can use in the first five minutes of a page and the first five hours of an incident. The manual covers the operator's toolkit (Chapter 2), the routine operations that keep the platform healthy (Chapter 3), the deployment operations that change the platform (Chapter 4), the monitoring and alerting that surface the platform's state (Chapter 5), the incident-response runbooks for the seven most likely SEV-1/2 scenarios and one SEV-3 investigation (Chapter 6), the break-glass procedures for when no other path exists (Chapter 7), the disaster-recovery procedures for when the primary region is unrecoverable (Chapter 8), and the operator-health policies that keep the rotation sustainable (Chapter 9).

The manual's spine is the Five-A pattern (Acknowledge, Assess, Act, Alert, Ascertain) introduced in Playbook Ch 13 §13.8 and the blameless postmortem culture introduced in Playbook Ch 16 §16.7. Together, these are the operator's discipline: respond fast, mitigate before investigating, communicate visibly, and learn from every incident without blaming the humans who responded to it.

11.2 Implications for the Operator's Daily Life

For the operator's daily life, the manual implies:

- **Daily readiness —** the operator SHALL verify the toolkit at the start of every shift (Appendix A). A missing or broken tool is a shift-readiness failure.
- **Daily routine —** the operator SHALL run the daily constitutional health check and the daily certification coverage check. These are not optional; they are the morning vitals.
- **Deployment discipline —** the operator SHALL observe every canary stage of every deployment and SHALL rollback at the first SLO regression. Rollback is the default; investigation comes after.
- **Incident discipline —** the operator SHALL follow the runbook step-by-step during an incident. Deviation from the runbook is judgement, and judgement SHALL be recorded in the incident timeline.
- **Escalation discipline —** the operator SHALL page up at the first doubt. There is no penalty for escalating; there IS a penalty for not escalating.
- **Health discipline —** the operator SHALL take the cooldown after a SEV-1. The cooldown is recovery, not punishment; an operator who skips it is an operator who will burn out.
- **Learning discipline —** the operator SHALL file the blameless postmortem within 5 business days of a SEV-1/2 and SHALL track its action items to closure.

11.3 Future Evolution

This manual will evolve. Three directions are anticipated:

11.3.1 AI-Assisted Incident Response

An AI assistant trained on this manual, the Playbook, the Constitution, and the historical incident archive could provide the operator with a real-time suggested-runbook during an incident: given the alert payload, the assistant proposes the most likely runbook, surfaces the relevant steps, and drafts the communication updates. The operator remains the decision-maker; the assistant reduces the cognitive load. This is anticipated within 12–18 months of RC2 launch.

11.3.2 Predictive Alerting

The platform's anomaly-detection capability (currently SEV-3, RB-6.9) is expected to evolve toward predictive alerting: alerting the operator BEFORE the SLO burns, before the invariant turns red, based on leading indicators (latency percentile drift, error-rate trend, dependency-health degradation). Predictive alerting shifts the operator from reactive to preventive, which is the SRE discipline's long-term direction (SRE Book, sre.google/sre-book/monitoring-distributed-systems).

11.3.3 Runbook-as-Code

The runbooks in this manual are prose. A future evolution is runbook-as-code: each runbook SHALL be machine-executable, with the operator's role reduced to authorization (the operator approves each automated step). This is the direction of the SRE industry broadly (e.g., automated remediation, runbook automation platforms). The platform SHALL adopt runbook-as-code incrementally, starting with the highest-frequency runbooks (RB-6.5 Crossref, RB-6.6 SLO burn) and expanding as confidence grows. The operator's judgement SHALL remain in the loop for SEV-1 and break-glass runbooks.

Appendices

Appendix A — The On-Call Handoff Checklist

The handoff checklist SHALL be completed at every rotation handoff (typically 10:00 UTC every Monday). The outgoing primary SHALL walk the incoming primary through every item; the incoming primary SHALL confirm understanding of each item before the pager is transferred. The handoff SHALL NOT be considered complete until all 15 items are checked.

Handoff Checklist (15 items)

☐ **H-01** Open incidents reviewed; each has a current-state summary, mitigation in effect, next planned action, and war-room link.

☐ **H-02** Pending alerts from last 24h reviewed; each closed (with rationale) or carried forward as a watch item.

☐ **H-03** In-flight deployments identified; for each: stage, observation window remaining, rollback trigger conditions, owning Release Manager.

☐ **H-04** Scheduled jobs in next 24h identified (backups, reconciliation, DOI deposits, preservation ticks); for each: expected run time and failure-mode response.

☐ **H-05** Break-glass credential inventory confirmed; credentials present, not expired, not checked out.

☐ **H-06** Constitutional Health dashboard confirmed green at handoff moment; every invariant panel verified.

☐ **H-07** SLO dashboard burn rates reviewed; any SLO above 1x burn carried forward with the watch note.

☐ **H-08** Audit chain dashboard confirmed green; INV-A-03 cryptographic verification last-run timestamp within 1 hour.

☐ **H-09** Traceability graph dashboard confirmed green; last integrity-check run timestamp within 24 hours.

☐ **H-10** Self-audit report from previous night reviewed; every finding triaged (file-and-forget / watch / ticket / escalate).

☐ **H-11** Anomaly watch list (from anomaly-detection job) handed off with owner and next-action for each anomaly.

☐ **H-12** Open CC-failure tickets reviewed; each has an owner and a next-action.

☐ **H-13** Upcoming maintenance windows in next 24h identified; operator is aware and alert-silences are scheduled.

☐ **H-14** Test page acknowledged by incoming primary on their device; PagerDuty routing confirmed.

☐ **H-15** Rotation log entry recorded: timestamp, incoming primary, outgoing primary, carried-forward items, anomalies noted.

Appendix B — SEV-1 Incident Communication Template

The IC SHALL use this template for the initial SEV-1 communication (to #ops, the war-room channel, and the external status page within 30 minutes of user-visible impact). The template SHALL NOT be deviated from in structure; the bracketed fields SHALL be filled in.

```
INCIDENT:                [INC-YYYY-NNNN]
SEVERITY:                SEV-1
TITLE:                   [One-line summary, e.g., 'INV-I-02 duplicate DOI minted']
STATUS:                  [Investigating | Mitigating | Verifying | Resolved]
START:                   [YYYY-MM-DD HH:MM UTC]
IC:                      [@on-call-handle]
WAR ROOM:                [#incident-INC-YYYY-NNNN, bridge URL]

IMPACT:                  [User-visible impact, e.g., 'New articles cannot be published; existing articles remain readable.']
INVOLVED:                [Affected services, subsystems, integrations]
MITIGATION:              [What is being done, e.g., 'Publication queue frozen; duplicate DOI merge in progress.']
ETA:                     [Expected next update time, e.g., 'Next update in 30 minutes']

CSA NOTIFIED:            [Yes / No, with timestamp]
STATUS PAGE UPDATED:     [Yes / No, with timestamp]

NEXT UPDATE:             [YYYY-MM-DD HH:MM UTC]
```

The IC SHALL post an update using the same template every 30 minutes during the active incident, at minimum. The IC SHALL update the external status page every 60 minutes during user-visible impact.

Appendix C — Blameless Postmortem Template

Every SEV-1 and SEV-2 SHALL have a blameless postmortem filed within 5 business days of resolution. The postmortem SHALL follow this template. The postmortem SHALL NOT name individuals as causes; it SHALL name behaviours, decisions, and systemic conditions.

```
POSTMORTEM:              [INC-YYYY-NNNN]
SEVERITY:                [SEV-1 | SEV-2]
TITLE:                   [One-line summary]
AUTHOR:                  [IC or scribe]
REVIEWERS:               [Tech Lead, CSA, owning-team lead]
DATE:                    [YYYY-MM-DD]

1. SUMMARY
   [2-3 paragraph neutral summary: what happened, what was the impact, how it was
```

resolved. No blame.]

2. TIMELINE
[All times in UTC. From first alert to resolution. Include detection, acknowledgement, declaration, mitigation attempts, resolution, communication milestones.]

3. IMPACT
[User-visible impact, duration, number of users affected, scholarly-record impact, SLO budget consumed.]

4. ROOT CAUSE
[The fundamental reason the incident occurred. Not the proximate cause; the systemic reason the system was vulnerable to the proximate cause. Ask 'why?' five times.]

5. CONTRIBUTING FACTORS
[Conditions that made the root cause possible or worsened the impact: missing tests, missing alerts, runbook gaps, fatigue, tooling failures.]

6. WHAT WENT WELL
[Things that worked: fast acknowledgement, runbook followed correctly, good communication, fast rollback.]

7. WHAT WENT POORLY
[Things that did not work: runbook gap, slow escalation, tooling failure, communication delay.]

8. WHERE WE GOT LUCKY
[Things that could have been worse but were not, by chance. These are the next incidents waiting to happen.]

9. ACTION ITEMS
[Each action item: ID, description, owner, deadline, ticket URL. Action items SHALL be tracked to closure at the monthly operations review.]

10. LESSONS LEARNED
[What the platform should learn from this incident. Not action items (those are above); the broader lessons.]

11. POSTMORTEM REVIEW
[Reviewers' sign-off, with date. The postmortem is not closed until all reviewers have signed off.]

Appendix D — Break-Glass Request and Audit Form

Every break-glass invocation SHALL be recorded on this form. The form SHALL be filed in the break-glass log within 1 hour of credential check-in and SHALL be reviewed by the Tech Lead within 24 hours.

BREAK-GLASS INVOCATION: [BG-YYYY-NNNN]
DATE: [YYYY-MM-DD HH:MM UTC]

1. OPERATOR: [Name, SSO identity]
2. JUSTIFICATION: [SEV-1 with no other path / CI/CD outage during incident. Detail the specific situation.]
3. CSA APPROVER: [Name; Slack message timestamp; screenshot URL]
4. CREDENTIAL CHECKED OUT: [Vault path; checkout timestamp]
5. CREDENTIAL CHECKED IN: [Check-in timestamp; rotation confirmed]
6. DURATION: [Minutes; SHALL be ≤ 30]
7. ACTIONS TAKEN: [Every action performed with the credential, in order, with timestamps. SHALL match the audit-log entries.]
8. RELATED INCIDENT: [INC-YYYY-NNNN; war-room URL]
9. OPERATOR SELF-ASSESSMENT: [Did the break-glass achieve the intended mitigation? Was the justification valid in retrospect?]
10. TECH-LEAD REVIEW: [Judgment: justified / unjustified. Rationale. Date. Reviewer.]
11. FOLLOW-UP ACTION ITEMS: [What changes prevent the next invocation? Owner, deadline, ticket URL.]

Appendix E — Operator Command Quick-Reference

A one-page table of the most-used commands. The operator SHALL keep this table printed or pinned for rapid reference during an incident.

Task	Command
List pods in production	kubectl --context prod-eu -n opus-publica get pods
Tail a pod's logs	kubectl --context prod-eu -n opus-publica logs -f <pod>
Describe a pod (events, state)	kubectl --context prod-eu -n opus-publica describe pod <pod>
Scale a deployment	kubectl --context prod-eu -n opus-publica scale deploy/<svc> --replicas=N
Rollback a release (via Git revert)	git revert <merge> && git push # Flux auto-reconciles
Suspend Flux (freeze deploys)	flux --context prod-eu suspend kustomization opus-publica
Resume Flux	flux --context prod-eu resume kustomization opus-publica
Trigger a manual CronJob	kubectl --context prod-eu -n opus-publica create job --from=cronjob/<name> <name>-manual-$(date +%s)
Set a feature flag (rollout %)	opa-flag set --service=<svc> --name=<flag> --rollout=<N>%
Verify container signature	cosign verify --key prod.pub <image>

Search Rekor for an artifact	rekor-cli search --artifact <hash>
Run an OPA invariant check	opa eval -d policies/ -i input/<check>.json 'data.opus.constitutional.invariants'
Check out break-glass credential	vault kv get -field=<field> secret/opus/break-glass/<svc>
Check break-glass credential back in (rotate)	vault kv put secret/opus/break-glass/<svc> <field>=<new-value>
Watch a GitHub Actions run	gh run watch
Merge a PR (squash)	gh pr merge <PR> --squash --delete-branch
Run the audit-chain-verify job manually	kubectl --context prod-eu -n opus-publica create job --from=cronjob/audit-chain-verify audit-chain-verify-manual-$(date +%s)
Promote DB read-replica (DR only)	aws rds promote-read-replica --db-instance-identifier opus-secondary
Switch DNS (DR only)	aws route53 change-resource-record-sets --hosted-zone-id <zone> --change-batch file://dns-failover.json
Open a war-room channel	Slack: /incident declare # opens #incident-INC-YYYY-NNNN

Appendix F — References

The following references are the authoritative sources behind this manual. Where this manual is silent, the operator SHALL consult the reference.

Reference	Citation	URL / Location
Google SRE Book	Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). Site Reliability Engineering: How Google Runs Production Systems.	https://sre.google/sre-book/table-of-contents/
Google SRE Workbook	Beyer, B., et al. (2018). The Site Reliability Workbook.	https://sre.google/workbook/table-of-contents/
NIST SP 800-61 Rev. 2	Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012). Computer Security Incident Handling Guide.	https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final
RFC 2119	Bradner, S. (1997). Key words for use in RFCs to Indicate Requirement Levels.	https://www.rfc-editor.org/rfc/rfc2119

Opus Publica Engineering Constitution Vol I	Opus Publica internal. RC1 Engineering Constitution v1.0.	Internal repository: /constitution/vol-1
Opus Publica Engineering Constitution Vol II	Opus Publica internal. RC1 Engineering Constitution v1.0.	Internal repository: /constitution/vol-2
Opus Publica Engineering Playbook Ch 13	Opus Publica internal. On-Call and Operational Ownership.	Internal repository: /playbook/ch-13
Opus Publica Engineering Playbook Ch 14	Opus Publica internal. Operational Governance Execution.	Internal repository: /playbook/ch-14
Opus Publica Engineering Playbook Ch 15	Opus Publica internal. SLO Management.	Internal repository: /playbook/ch-15
Opus Publica Engineering Playbook Ch 16	Opus Publica internal. Incident Command.	Internal repository: /playbook/ch-16
Opus Publica Certification Checklist	Opus Publica internal. CC-INV-X-NN criteria.	Internal repository: /certification-checklist
Opus Publica Volume II Ch 26 (Invariants)	Opus Publica internal. INV-I, INV-A, INV-T, INV-D, INV-S, INV-L invariants catalog.	Internal repository: /constitution/vol-2/ch-26
Opus Publica Volume II Ch 32 (SLO Book)	Opus Publica internal. Ten SLOs, three SLAs, error-budget policy.	Internal repository: /constitution/vol-2/ch-32
Opus Publica Volume II Ch 33 (Disaster Recovery)	Opus Publica internal. RPO 15 min, RTO 1 hour, DR plan.	Internal repository: /constitution/vol-2/ch-33
Opus Publica Volume II Ch 34 (Operational Governance)	Opus Publica internal. Authority matrix OP-GOV-001…OP-GOV-014.	Internal repository: /constitution/vol-2/ch-34